COMPLETE PROJECT EXPLANATION AND UNDERSTANDING GUIDE

CNN-LSTM Based Network Traffic Forecasting

with proactive congestion avoidance

How it works, why every decision was made, what the results show,
and how to explain it in a presentation, viva or interview.

Project	B.Tech final-year project: short-term forecasting of backbone network traffic
Datasets	Abilene (144 flows, 5-min, 2004) · GÉANT (529 flows, 15-min, 2005)
Model	Residual CNN-LSTM (TensorFlow / Keras), compared with persistence and ARIMA
Covers	Code and results as of 19 October 2026 (common.py + 10 scripts, metrics.csv, run logs)
Prepared	20 October 2026, from the project files in traffic_project/

Every number in this guide comes from the project's own logs, CSV and saved models, or is explicitly marked as computed for this guide. Nothing is invented; unverifiable points are stated as such.

Contents

How to Use This Guide . . . 6

Which version this covers . . . 6

How to read the evidence labels . . . 6

Inconsistencies and gaps found while checking the files . . . 6

1. Project Overview . . . 8

1.1 What the project is, in one paragraph . . . 8

1.2 The problem it addresses . . . 8

1.3 Objectives . . . 8

1.4 The expected and the actual output . . . 9

1.5 A real-world analogy . . . 9

1.6 What makes it useful or different . . . 9

2. Core Concepts and Fundamentals . . . 10

2.1 Backbone network, routers and links . . . 10

2.2 Origin–destination (OD) flow and the traffic matrix . . . 10

2.3 Routing matrix . . . 10

2.4 Time series, sampling interval and horizon . . . 11

2.5 Persistence baseline . . . 11

2.6 ARIMA . . . 11

2.7 Supervised learning, windows and samples . . . 12

2.8 Chronological train / validation / test split . . . 12

2.9 Scaling (log1p + per-flow min–max) . . . 12

2.10 Neural-network building blocks . . . 13

2.11 Residual (skip) connection . . . 14

2.12 Link utilisation and congestion . . . 14

2.13 Error metrics (summary; formulas in chapter 7) . . . 14

2.14 Seeds and reproducibility . . . 15

3. Complete Project Architecture . . . 16

3.1 Components and their responsibilities . . . 16

3.2 How components communicate . . . 17

3.3 What enters and leaves each stage . . . 17

3.4 Technologies used at each layer (details in chapter 8) . . . 18

4. Complete Working and Execution Flow . . . 19

4.1 What happens when a script starts . . . 19

4.2 Inputs the system receives . . . 19
4.3 Processing, step by step (Abilene, Phase 1, 01_data.py) . . . 19
4.4 Model training and prediction (Phases 3–5) . . . 20
4.5 Link loads and warnings (Phase 6, 04_analysis.py) . . . 20
4.6 What happens with errors, unexpected input and exceptional cases . . . 20
4.7 What the user finally sees . . . 21

5. Complete Logic Explanation . . . 22
5.1 Choosing the measured column (realOD) . . . 22
5.2 The missing-data policy (cleaning) . . . 22
5.3 Windowing . . . 23
5.4 Chronological split . . . 23
5.5 Scaling and its inverse . . . 24
5.6 Building the model: plain vs residual . . . 24
5.7 Training: loss choice, optimiser and stopping rules . . . 25
5.8 Prediction with a physical cap . . . 26
5.9 Evaluation logic . . . 26
5.10 ARIMA procedure (02_baselines.py) . . . 26
5.11 Tuning rule and seeds (03c_tuning.py) . . . 26
5.12 Horizons and ablations (03d_horizons.py) . . . 27
5.13 Congestion-warning logic (04_analysis.py) . . . 27
5.14 Approximate routing for GÉANT . . . 28
5.15 Error-analysis definitions (05_errors.py) . . . 28
5.16 Logging and the results page . . . 28

6. Source Code Explanation . . . 29
6.1 Folder structure . . . 29
6.2 How a script is organised . . . 29
6.3 The important functions, in the required format . . . 29
6.4 Tracing one complete run through the code . . . 33
6.5 Non-obvious statements worth knowing . . . 33

7. Algorithms and Mathematical Logic . . . 34
7.1 Unit conversion: raw units → Mbit/s . . . 34
7.2 Scaling: log1p + per-flow min–max . . . 34
7.3 Window count . . . 34
7.4 MAE, RMSE, MAPE and coverage, worked by hand . . . 35
7.5 Improvement over persistence . . . 35
7.6 Error as a share of traffic (WAPE), computed for this guide . . . 35
7.7 Layer arithmetic: where the 143,608 parameters come from . . . 35

7.8 The LSTM equations . . . 36
7.9 Training algorithms . . . 36
7.10 The original-units loss (mae_raw) . . . 36
7.11 Link utilisation . . . 36
7.12 Precision, recall, F1 . . . 37
7.13 Damping (how much of a move is forecast) . . . 37
7.14 Floyd–Warshall and shortest-path counting (GÉANT routing) . . . 37
7.15 ARIMA in one equation . . . 37
7.16 Implemented vs only discussed . . . 37

8. Technologies, Tools and Dependencies . . . 38
8.1 How they interact . . . 38
8.2 File formats . . . 38
8.3 Installation and configuration . . . 38

9. Data Flow and Worked Examples . . . 40
Example 1: one 5-minute forecast for the busiest flow . . . 40
Example 2: a congestion onset on the busiest link, 30 minutes ahead . . . 41
Example 3: how the training state changes over time . . . 42

10. Testing, Evaluation and Results . . . 43
10.1 How the project is tested . . . 43
10.2 Important test scenarios . . . 43
10.3 Measured results (all from the project's logs) . . . 43
10.4 What the results demonstrate . . . 45
10.5 What the results do NOT prove . . . 45
10.6 Known bugs, limitations and unverified claims . . . 46

11. Novelty and Design Decisions . . . 47
11.1 What is genuinely distinctive . . . 47
11.2 Existing methods it uses . . . 47
11.3 Key design decisions and their trade-offs . . . 47
11.4 Implemented vs future work . . . 48

12. Limitations and Future Improvements . . . 49
12.1 Current limitations (existing functionality) . . . 49
12.2 Assumptions that may not hold in the real world . . . 49
12.3 Suggested improvements (not implemented) . . . 49

13. Important Questions and Answers . . . 51
13.1 Basic . . . 51

13.2 Concepts . . . 51
13.3 Architecture . . . 52
13.4 Implementation and code decisions . . . 52
13.5 Results . . . 53
13.6 Testing and metrics . . . 53
13.7 Novelty . . . 53
13.8 Limitations and next steps . . . 53
13.9 Challenging questions . . . 54

14. Quick Revision . . . 55

14.1 One-page summary . . . 55
14.2 The complete flow in numbered steps . . . 55
14.3 Concepts to remember . . . 55
14.4 Key formulas and metrics . . . 56
14.5 Component → responsibility . . . 56
14.6 Most important functions . . . 56
14.7 Novelty in two sentences . . . 57
14.8 The two-minute explanation . . . 57

Glossary . . . 58

References to project sources . . . 59

How to Use This Guide

This guide explains the project **CNN-LSTM Based Network Traffic Forecasting (with proactive congestion avoidance)** from first principles. It was written by reading every source file, the project log, the run logs and the saved results, and every number in it is traced to one of those sources.

Which version this covers

Item	Version described
Code	`common.py` and the 10 scripts in `scripts/` as they were on 19 Oct 2026 (latest edits: `05_errors.py` 17:21, `common.py` 16:22)
Results	`results/metrics.csv` (last row 19 Oct 2026 16:43), run logs `03c_run.log` … `05_run.log`, 21 figures in `results/figures/`
Trained models	about 100 `.keras` files in `models/`; the final ones are the `p4_*`, `p5_*` and `g_*` models (3 seeds each)
Documentation	`README.md`, `docs/PROJECT_LOG.md` (sections 1–18)

How to read the evidence labels

- **Measured**: a number produced by the project's own code and stored in its logs, CSV or figures.
- **Computed for this guide**: a number calculated while preparing this guide from the project's saved models and data with a separate read-only script (for example the worked examples and the 'error as % of traffic' figures). These are *not* yet in the project's report and are marked where used.
- **Interpretation**: an explanation of why a result happened. Where the project tested an explanation it is stated as a finding; where it did not, it is labelled as a hypothesis.
- **Proposed / future work**: something that has not been implemented.

Inconsistencies and gaps found while checking the files

These do not affect the results, but you should know about them before a review:

Where	What is inconsistent or missing
`docs/PROJECT_LOG.md`	Header says 'Last updated 8 October 2026' and the summary in section 2 still says 'No model yet clearly beats persistence'. Sections 14–18 (dated by the 19 October runs) show that models do beat persistence slightly. The summary is stale.
`README.md`	The layout table says `03_models.py` holds 'LSTM, CNN, CNN-LSTM, multi-step, ablations' and `04_analysis.py` holds 'GÉANT, peak-error analysis'. In fact these were split into `03b_hybrid`, `03c_tuning`, `03d_horizons`, `03e_geant` and `05_errors`.
`scripts/01_data.py`	Docstring says output goes to `results/reports/01_data.html`; the code actually writes `results/reports/01_data.json` and rebuilds `results/report.html`.
`scripts/03_models.py`	Docstring says 'Phases 3 to 5', but the file contains only Phase 3; Phases 4–5 live in the 03b/03c/03d scripts.
`requirements.txt`	Library names only, no versions pinned (the log notes this as a to-do). The versions actually installed are listed in chapter 8.
`docs/presentation/*.pptx`	The 10-slide deck is the 8 October mid-project review. It stops at Phase 3 ('2.8× worse than persistence') and marks congestion avoidance as future work.
`tempCodeRunnerFile.c/.cpp/.exe`	Unrelated C/C++ scratch files from the VS Code Code Runner extension; not part of the project.

Where	What is inconsistent or missing
Not verifiable	The time zone of the GÉANT timestamps is undocumented; the Abilene README dates are partly unreliable (duplicate days appear between weeks listed as 7 days apart). Training was not re-run for this guide: results come from the saved logs and models.

1. Project Overview

1.1 What the project is, in one paragraph

A backbone network, such as the US research network **Abilene**, is a set of routers joined by high-speed links. Every pair of routers exchanges data. For each pair, called an **origin–destination (OD) flow**, we know how many bytes were sent in every 5-minute interval. This project trains neural networks to **forecast every flow 5, 30 or 60 minutes ahead**, then converts those forecasts into **predicted load on each physical link**, so that a link that is about to become congested can be flagged *before* it happens. The main model is a **CNN-LSTM**: convolution layers read short patterns in the last hour of traffic, and LSTM layers model how those patterns evolve over time.

> **NOTE This is computer-network traffic, not road traffic**
> The 'traffic' is bytes moving between routers. 'Congestion' means a network link carrying a large share of its capacity, which causes queuing delay and packet loss.

1.2 The problem it addresses

- **Congestion is usually handled reactively.** Operators notice an overloaded link (from monitoring that reports the current load) and then reroute traffic or add capacity. By then users have already suffered delay or loss.
- **Traffic changes constantly.** Each flow rises and falls through the day: on Abilene the whole-network peak is about 1.5× the quietest point, and some flows burst from about 100 Mbit/s to over 6,000 Mbit/s.
- **A forecast would allow proactive action**: if you knew the load 30 minutes ahead, you could reroute or rate-limit in advance.

1.3 Objectives

#	Objective	Status
1	Prepare real backbone traffic data correctly (clean it, split it by time, scale it without leaking the future).	Done
2	Measure strong, simple baselines: persistence ('next = now') and ARIMA.	Done
3	Build and train LSTM-only, CNN-only and CNN-LSTM models; tune the CNN-LSTM.	Done
4	Forecast further ahead (30 and 60 minutes) and test which parts of the model matter (ablations).	Done
5	Turn flow forecasts into link loads and congestion warnings; compare with reacting to the current load.	Done
6	Check that the findings hold on a second, larger network (GÉANT, Europe).	Done
7	Analyse where and why the model gains or loses against persistence.	Done
8	Final report and presentation.	Not started (deck is the mid-project version)

1.4 The expected and the actual output

The system outputs, for every time step in the test period: (a) a forecast for every flow, (b) the forecast utilisation of every backbone link, and (c) a warning flag for every link whose forecast utilisation is at or above a threshold. All scores are written to `results/metrics.csv` and gathered with every figure on one page, `results/report.html`.

> **KEY POINT The honest headline result (measured)**
> The CNN-LSTM beats persistence on both networks, but only slightly: about **+0.3 to +1%** lower error at 5–15 minutes and up to **+2.7%** at 60 minutes (GÉANT). At link level it gives **no early-warning advantage** over simply reacting to the current load. The project's real value is a careful, reproducible evaluation that shows *why*: short-term traffic changes are mostly unpredictable from the last hour.

1.5 A real-world analogy

Think of a weather forecaster. The simplest forecast is 'tomorrow will be like today'. It is surprisingly accurate because weather changes slowly. A sophisticated model is only useful if it beats that simple rule. In this project, 'today' is the last 5-minute reading of each flow; that rule is called **persistence**. The CNN-LSTM is the sophisticated forecaster. The project measures carefully how much better than 'same as now' it really is.

The routing step is like a road map: if you forecast how many cars will travel from city A to city B, and you know which roads that trip uses, you can add up the forecast traffic on each road and predict which road will jam.

1.6 What makes it useful or different

- **A strong baseline, everywhere.** Every result is reported next to persistence. Many published studies compare deep models only with other deep models or with ARIMA.
- **Leak-free evaluation.** The data are split chronologically, the scaler is fitted on training data only, nothing is tuned on the test split, and windows never cross data gaps.
- **Errors in real units.** All metrics are computed in Mbit/s after undoing the scaling, not on 0–1 scaled values, where small errors look impressive.
- **End to end.** It goes beyond flow accuracy to link loads and warning quality, which is what an operator would actually use.
- **A second network.** The same design was retrained, with no retuning, on GÉANT (529 flows, 15-minute data).
- **Findings explained, not just reported.** For example, one bursty flow decides the model ranking on Abilene, and the model forecasts only 1–5% of each real change.

> **NOTE What it does not claim**
> It does not claim a new neural architecture. CNN-LSTMs for traffic forecasting already exist in the literature. See chapter 11 for what is and is not new.

2. Core Concepts and Fundamentals

This chapter defines every important idea before it is used. Each concept says what it is, why the project needs it, how it works, a small example, and how it connects to the rest.

2.1 Backbone network, routers and links

> **DEFINITION Backbone network**
> The high-capacity core of a network that connects regional networks. Abilene had 12 core routers (two in Atlanta) joined by 30 directed backbone links, mostly 9.92 Gbit/s.

- **Why needed:** it is the system being forecast.
- **How it works:** a packet enters at one router (ingress), travels over one or more links, and leaves at another router (egress). The routing protocol (OSPF on Abilene) decides the path.
- **Connects to:** the routing matrix (2.3) and link utilisation (2.12).

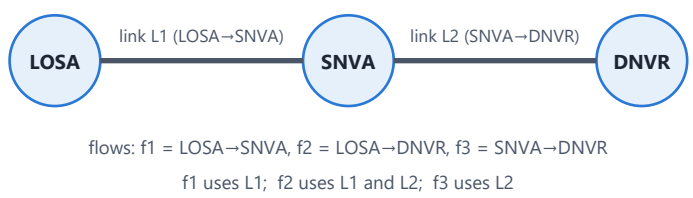
2.2 Origin–destination (OD) flow and the traffic matrix

> **DEFINITION OD flow / traffic matrix**
> An OD flow is all traffic that enters the network at router *i* and leaves at router *j*. The **traffic matrix** at one moment is the table of all OD flows; with 12 routers it is 12 × 12 = **144 flows** (including i = j, traffic entering and leaving at the same router).

- **Why needed:** flows are what the model forecasts. Forecasting per flow (not per link) means a forecast can be re-routed onto any link layout.
- **How it is stored:** flow number k = source × 12 + destination, with routers in the order of `ABILENE_NODES` in `common.py:56`. For example LOSA→CHIN is 7 × 12 + 2 = flow 86.
- **Example:** WASH→WASH averaged 260 Mbit/s in training; the smallest flows average well under 1 Mbit/s.

2.3 Routing matrix

> **DEFINITION Routing matrix A**
> A table with one row per link and one column per flow. Entry A[l, k] is the fraction of flow k that travels over link l (on Abilene always 0 or 1, because each flow follows exactly one path). Link loads are then a matrix product: **load = A @ flows**.



routing matrix A (rows = links, columns = flows)

	f1	f2	f3
L1	1	1	0
L2	0	1	1

load = A @ flows. With f = (100, 50, 30) Mbit/s:
L1 = 100 + 50 = 150, L2 = 50 + 30 = 80 Mbit/s
utilisation L1 = 150 / 9,920 = 1.5% of capacity

Abilene: A is 54 × 144 (rows 1–30 backbone links, 31–54 router entry/exit); every flow uses exactly one path.

Diagram: *Tiny example of a routing matrix with three routers, two links and three flows.*

- **In the project:** `load_abilene_routing()` reads `raw/A` (54 links × 144 flows; rows 1–30 are backbone links, 31–54 are router entry/exit points). For GÉANT there is no published routing, so `load_geant_routing()` builds an approximate one (shortest paths by hop count, split equally between equal-length paths).

2.4 Time series, sampling interval and horizon

> **DEFINITION Time series**
> A sequence of measurements taken at regular intervals. Here every flow is a time series with one value per **5 minutes** (Abilene) or **15 minutes** (GÉANT).

> **DEFINITION Horizon**
> How many steps ahead the forecast is made. Horizon 1 = 5 minutes ahead on Abilene; horizon 6 = 30 minutes; horizon 12 = 60 minutes. On GÉANT horizon 4 = 60 minutes.

- **Why it matters:** the further ahead, the less the current value tells you, so there is more room for a model to help, but the problem is harder. Each horizon has its own data folder and its own persistence reference.

2.5 Persistence baseline

> **DEFINITION Persistence (naive forecast)**
> Predict that the next value equals the most recent observed value: forecast(t + h) = value(t). It needs no training.

`common.py:250`

```python
def persistence(X):
    """Persistence baseline: predict that the next value equals the last input value."""
    return X[:, -1, :]
```

- **Why needed:** at 5-minute steps traffic changes little between readings, so persistence is very hard to beat. A model that cannot beat it is useless in practice.
- **Example:** if WASH→WASH carried 334.9 Mbit/s in the last 5 minutes, persistence forecasts 334.9 Mbit/s for the next 5 minutes.
- **Connects to:** it is the reference for every result, and it is also the starting point of the residual models (2.11).

2.6 ARIMA

> **DEFINITION ARIMA(p, d, q)**
> A classical statistical forecaster. **AR** (p): the next value is a weighted sum of the last p values. **I** (d): model the change between steps instead of the level if d = 1. **MA** (q): also use the last q forecast errors.

- **Why needed:** it is the standard non-neural comparison in forecasting research.
- **How it was used:** one model per flow for the 5 busiest flows, 7 candidate orders, order chosen on validation error (`02_baselines.py`, `ORDERS`).
- **Result:** 2.8% worse than persistence on test. Its smoothed forecasts react late to bursts.

2.7 Supervised learning, windows and samples

> **DEFINITION Supervised sample**
> A pair (input X, correct answer y). The model learns a function from X to y by seeing many such pairs.

A time series is turned into samples by a **sliding window**: the input is the last 12 readings of all 144 flows (1 hour) and the answer is the reading of all 144 flows h steps later. Then the window slides forward by one step.

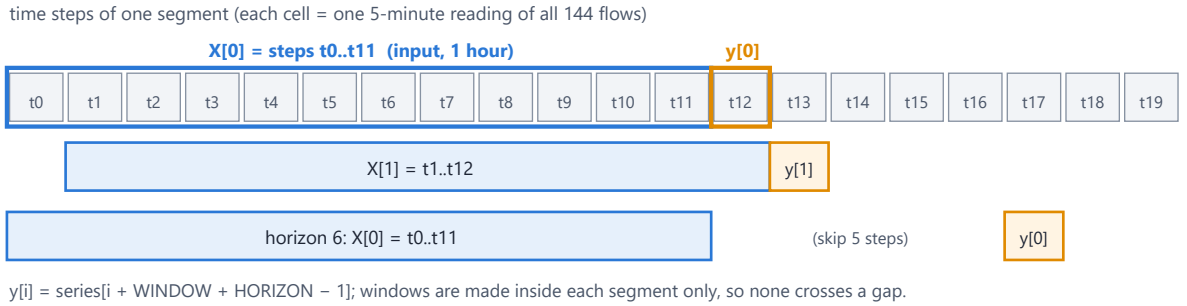


Diagram: *Sliding windows (`make_windows`, common.py:164). Orange = target; blue = input.*

2.8 Chronological train / validation / test split

> **DEFINITION Three splits**
> **Training**: used to learn the weights. **Validation**: used to decide when to stop training and which settings to keep. **Test**: used once, at the end, to report results.

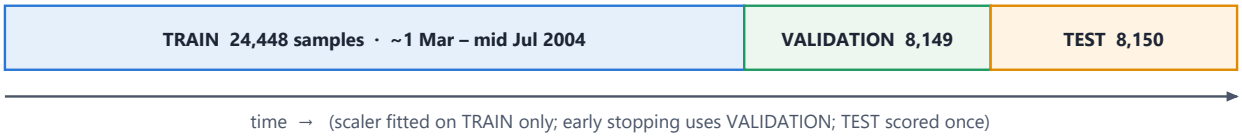


Diagram: *The chronological 60/20/20 split on Abilene.*

- **Why chronological, never random:** in real use you always predict the future from the past. A random split would put samples from after the test period into training, and neighbouring windows overlap in 11 of 12 steps, so the test would effectively be seen during training.
- **Why three splits:** if settings were chosen on the test split, the test score would be optimistic.

2.9 Scaling (log1p + per-flow min–max)

> **DEFINITION Feature scaling**
> Transforming values to a range a neural network learns well from. Here: z = (log(1 + x) − min) / (max − min), with min and max of log(1 + x) measured per flow on the **training inputs only**.

- **Why log:** values span from 0 to billions of raw units; the log compresses that range so one huge flow does not dominate. The '+1' keeps 0 valid (log(1 + 0) = 0).
- **Why per flow:** each flow is stretched to its own 0..1 range, so small flows are visible to the network.
- **Why train only:** measuring min/max on all data would leak information about the test period.
- **Inverse:** x = exp(z · (max − min) + min) − 1, applied before every metric.
- **Code:** class `FlowScaler`, `common.py:96`. Worked example in chapter 7.

2.10 Neural-network building blocks

Dense (fully connected) layer

Every output is a weighted sum of all inputs plus a bias: y = W·x + b. The final layer here is Dense(144): one output per flow.

1-D convolution (Conv1D), kernel, filter and causal padding

> **DEFINITION Conv1D**
> Slides a small set of weights (a **kernel** of size 3 time steps) along the time axis and computes a weighted sum at every position. Each of the 64 **filters** learns to detect one short pattern, such as 'a sudden rise', across all 144 flows at once.

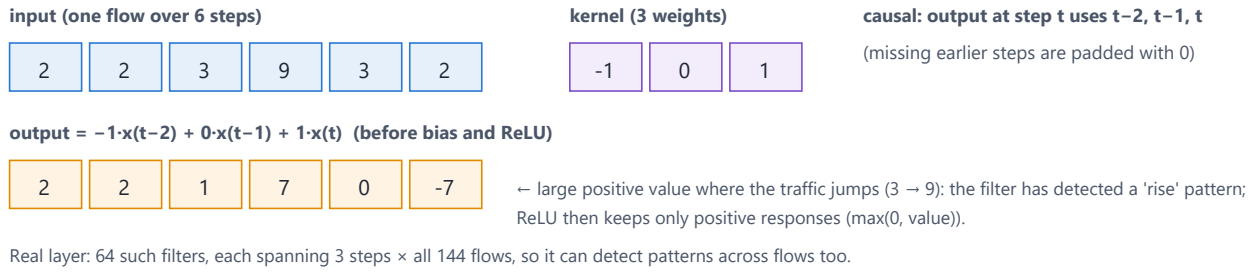


Diagram: *How one causal Conv1D filter responds to a jump in traffic (illustrative numbers).*

- **Causal padding** (padding='causal') means the output at step t uses only steps t, t−1, t−2, never later steps. Without it the CNN could peek at the end of the window.
- **ReLU** activation: max(0, v). It keeps positive responses and lets the network represent non-linear patterns.

Max pooling

MaxPooling1D(2) keeps the larger of every two consecutive time steps, halving the sequence from 12 to 6 steps. It keeps the strongest pattern responses and makes the LSTM's job shorter.

LSTM (Long Short-Term Memory)

> **DEFINITION LSTM**
> A recurrent layer that reads a sequence one step at a time and keeps an internal memory. Learned 'gates' (numbers between 0 and 1) decide what to forget, what to write and what to output at each step, which lets it keep information over many steps without the signal fading.

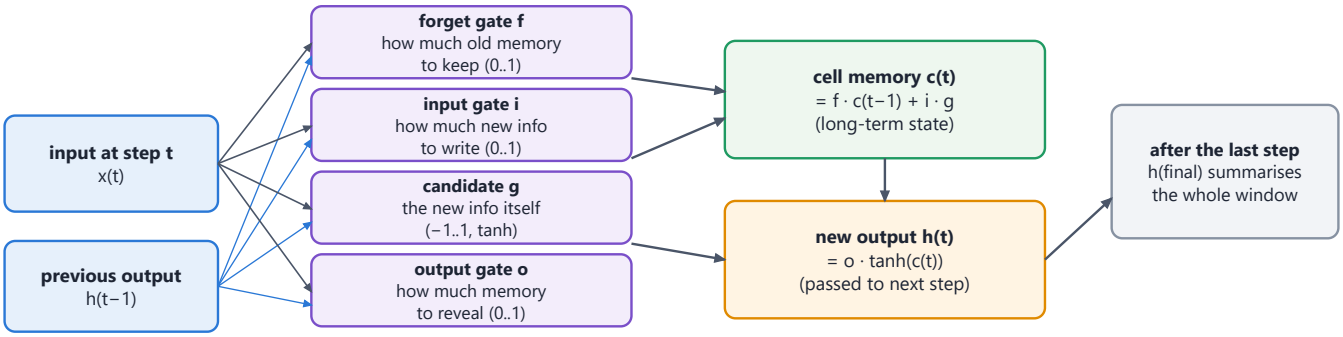


Diagram: *What happens inside one LSTM step (simplified).*

- The first LSTM (100 units) returns its output at every one of the 6 steps; the second (50 units) returns only its final output, a 50-number summary of the whole hour.

Dropout

During training, 20% of the 50 summary values are randomly set to zero at each update. This forces the network not to rely on any single unit and reduces memorising (overfitting). It is switched off when predicting.

2.11 Residual (skip) connection

DEFINITION Residual forecast

Instead of predicting the value itself, the network predicts a **correction** that is added to the last observed value: forecast = last value + correction. The correction layer's weights start at zero, so an untrained residual model is exactly persistence.

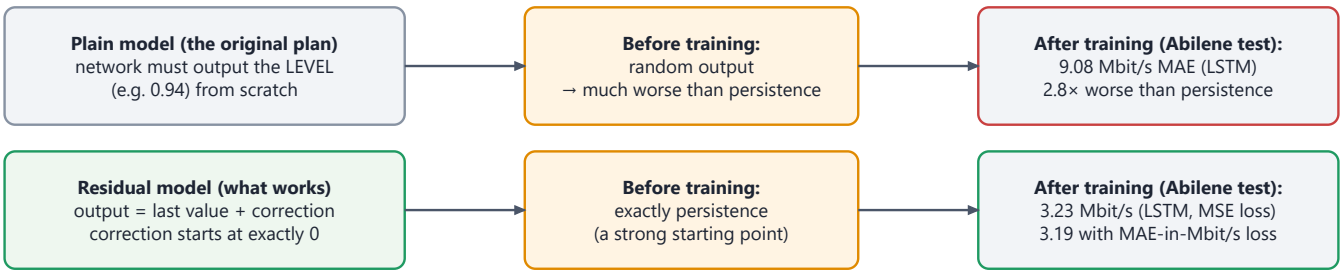


Diagram: *Why the residual form was essential (numbers measured on the Abilene test split, 5 minutes ahead).*

- **Why needed:** the plain models had to learn the level of each flow from scratch, and in practice pulled forecasts toward typical values. They ended 2.8–3.4× worse than persistence.
- **Code:** `_output(..., residual=True)`, `common.py:703`, using Keras Cropping1D (keep the last step), Flatten and Add.

2.12 Link utilisation and congestion

DEFINITION Utilisation

Load on a link divided by its capacity, e.g. 4.96 Gbit/s on a 9.92 Gbit/s link = 50%.

DEFINITION Congestion event / onset

A link-step is 'congested' if utilisation ≥ threshold. An **onset** is the first congested step after a non-congested one, the moment an early warning matters most.

- The plan's 80% threshold could not be used, because no Abilene link ever reaches 80% (the busiest reaches 75%). The project used **50%**, fixed before seeing any warning results, with 40% and 60% as sensitivity checks.

2.13 Error metrics (summary; formulas in chapter 7)

Metric	Plain meaning	Why used
MAE	Average size of the error, in Mbit/s	Main metric; easy to read; the loss the final models train on
RMSE	Like MAE but large errors count more (squared)	Shows sensitivity to big misses (bursts)
MAPE (+ coverage)	Average error as a % of the true value, only where the true value ≥ 10% of the mean	Near-zero flows make % errors explode, so coverage says how much data was included

Metric	Plain meaning	Why used
% change vs persistence	$1 - \text{MAE}_{\text{model}} / \text{MAE}_{\text{persistence}}$; positive = better	The headline comparison
Precision / recall / F1	Quality of congestion warnings	Link-level evaluation
Damping	How much of each real change the model forecasts (0 = none, 1 = all)	Error analysis (Phase 7)

2.14 Seeds and reproducibility

> **DEFINITION** **Random seed**
> A starting number for the random-number generator. With the same seed, weight initialisation and batch shuffling are identical, so a run can be repeated exactly.

- `set_seeds()` seeds Python, NumPy and TensorFlow and turns on deterministic TensorFlow operations.
- Final models are trained with **three seeds (42, 43, 44)**. Because the gains are only about 1%, the spread between seeds (about 0.5–1%) shows what counts as a real difference.

3. Complete Project Architecture

The project is a **batch machine-learning pipeline**, not a running service. There is no server, database or user interface. Each stage is a Python script that reads files, computes, and writes files. All shared logic lives in one library, `common.py`.

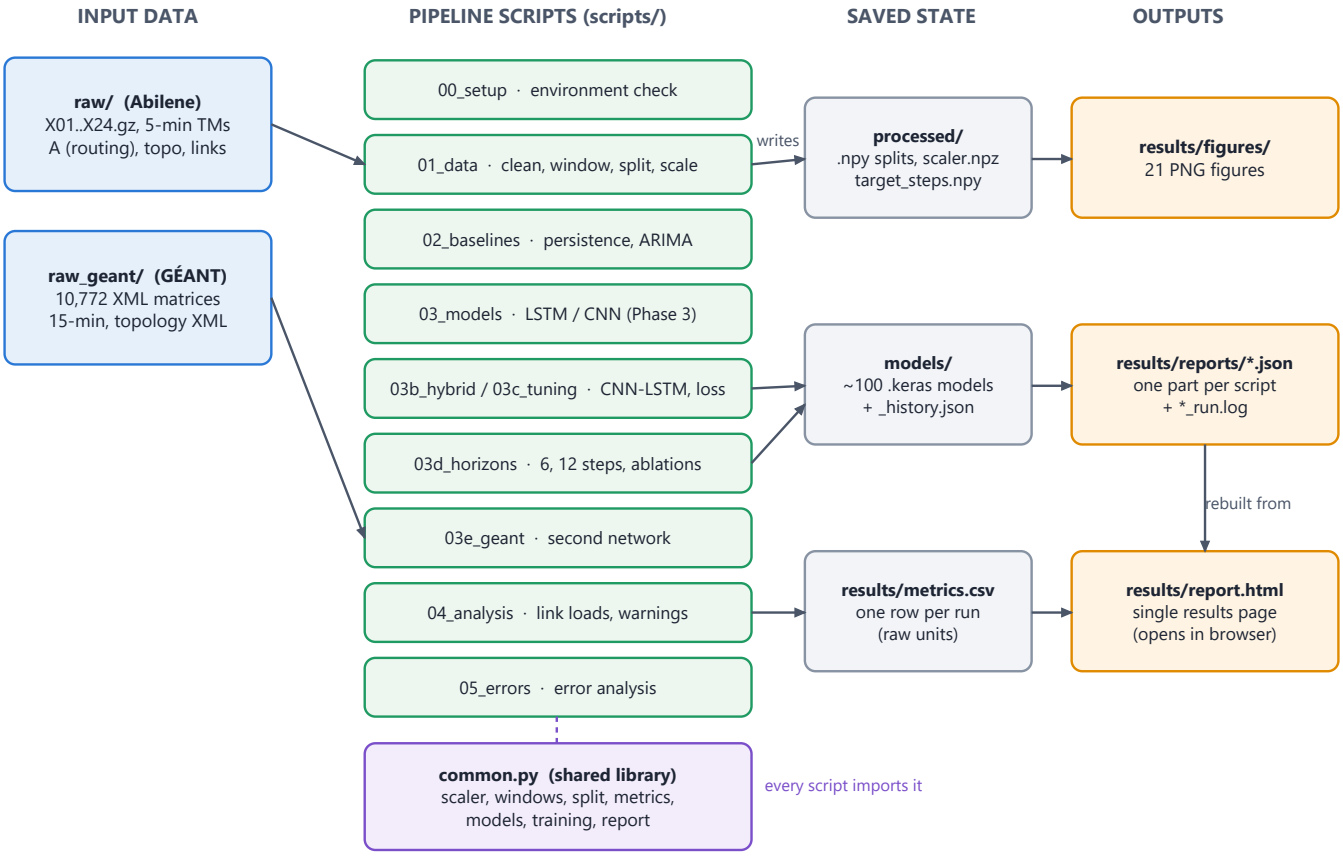


Diagram: *Architecture: data sources, pipeline scripts, saved state and outputs.*

3.1 Components and their responsibilities

Component	Purpose	Reads	Writes
`raw/`	Original Abilene data: 24 weekly traffic-matrix files, routing matrix A, link list, topology with capacities	–	–
`raw_geant/`	Original GÉANT (TOTEM) data: 10,772 XML traffic matrices and the anonymised topology	–	–
`common.py`	The only shared library: constants, scaler, windowing, splitting, cleaning rules, metrics, logging, routing, model builders, training, prediction, results page, self-test	raw/, processed/	processed/, models/, results/
`00_setup.py`	Environment check: library versions, self_test(), builds the planned model (143,608 parameters)	–	screen
`01_data.py`	Phase 1: load, choose the realOD column, clean, segment, window, split, scale, save, check	raw/	processed/*.npy, scaler.npz
`02_baselines.py`	Phase 2: persistence on all flows; ARIMA on the 5 busiest	processed/, raw/	metrics.csv, figures
`03_models.py`	Phase 3: LSTM-only and CNN-only, plain and residual	processed/	models/, metrics.csv

Component	Purpose	Reads	Writes
`03b_hybrid.py`	Phase 4 step 1: CNN-LSTM; compare training losses (MSE, MAE, MAE in original units)	processed/	models/, metrics.csv
`03c_tuning.py`	Phase 4: one-at-a-time tuning; 3 seeds CNN-LSTM vs LSTM-only	processed/	models/, metrics.csv
`03d_horizons.py`	Phase 5: horizons 1, 6, 12; ablations	processed/, h6/, h12/	models/, metrics.csv
`03e_geant.py`	Phase 6: GÉANT data, models at 15 and 60 min, approximate link level	raw_geant/	processed/geant/, models/g_*
`04_analysis.py`	Phase 6: Abilene link loads and congestion warnings	models/, processed/, raw/A	figures, report
`05_errors.py`	Phase 7: where the model gains/loses; damping; peaks; one-flow effect	models/, processed/	figures, report
`results/`	metrics.csv (one row per run), figures/, reports/*.json, *_run.log, report.html	–	–

3.2 How components communicate

Components never call each other directly. They communicate through **files with a fixed format**, the **data contract** written at the top of `common.py`:

Artifact	Format	Produced by	Consumed by
X_train / X_val / X_test	float32 array (samples, 12, flows), scaled 0..1	_prepare_segments	every model script
y_train / y_val / y_test	float32 array (samples, flows), scaled	_prepare_segments	every model script
scaler.npz	per-flow min and max of log1p(x) on training inputs	FlowScaler.save	loss, predict, evaluate
target_steps.npy	timeline index of each sample's target	_prepare_segments	ARIMA alignment, onsets, labels
name.keras + *name*_history.json	saved model + loss per epoch	fit_or_load	later scripts (no retraining)
metrics.csv	timestamp, name, dataset, horizon, mae, rmse, mape, mape_coverage, notes (raw units)	log_result	report page, comparisons
reports/*script*.json	captured printout + figures of one script	Report.finish	build_results_page

3.3 What enters and leaves each stage

Stage	In	Out
Load	24 gzip text files, each 2016 rows × 720 numbers	array (24 weeks, 2016 steps, 144 flows), raw units
Clean	the weekly array	9 segments (start index, array T_i × 144) + removal reasons
Window + split + scale	segments	6 arrays + scaler + target index
Train	scaled train and validation arrays	a model whose weights minimise validation loss
Predict	scaled test inputs	scaled forecasts, clipped to the physical range
Evaluate	forecasts and truth, both converted back to raw units	MAE, RMSE, MAPE, coverage → metrics.csv

Stage	In	Out
Link level	flow forecasts (raw units)	utilisation per link and step → warnings → precision/recall/F1/onsets

3.4 Technologies used at each layer (details in chapter 8)

- **Python 3.13** in a project virtual environment (TensorFlow does not support 3.14).
- **NumPy** for all array work; **pandas** to read metrics.csv; **statsmodels** for ARIMA.
- **TensorFlow / Keras 2.21** for the neural networks.
- **matplotlib** for figures; a self-built HTML results page.
- **VS Code** with '# %%' cells (Interactive Window) instead of Jupyter notebooks.

4. Complete Working and Execution Flow

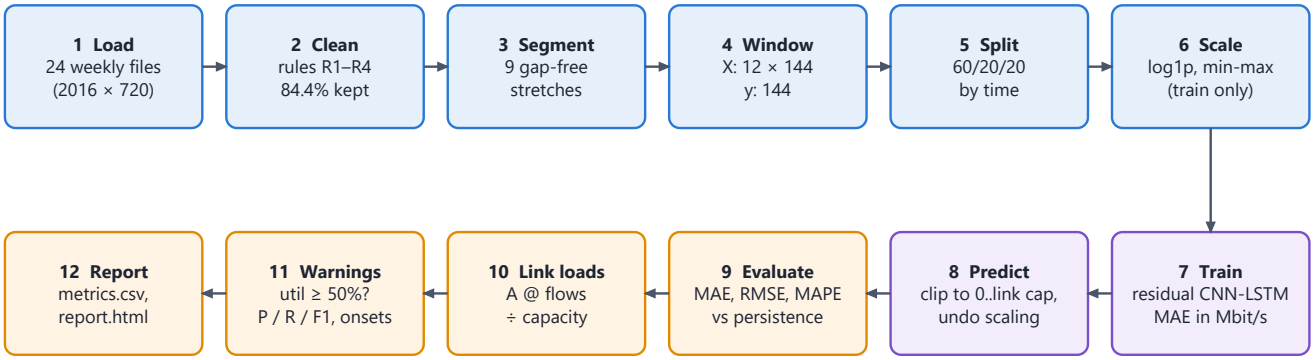


Diagram: *The end-to-end flow (Abilene). GÉANT follows the same steps with its own loader and cleaning rules.*

4.1 What happens when a script starts

1. The **header cell** walks up from the script's folder until it finds `common.py`, adds that folder to Python's import path, imports NumPy (and TensorFlow for model scripts), then **reloads** common.py so edits take effect in a running VS Code session.
2. Importing common.py runs `ensure_dirs()`, which creates raw/, processed/, models/, results/, figures/ and reports/ if they are missing.
3. A `Report` object is created. It replaces `sys.stdout` with a 'tee' that prints to the screen *and* records the text, and installs hooks so an error or normal exit still saves the script's part of the page.
4. `set_seeds()` fixes all random generators; `status()` prints what data and models already exist.

4.2 Inputs the system receives

Input	Content	Size
`raw/X01.gz … X24.gz`	Abilene, one week each: 2016 rows (5-min steps) × 720 numbers (144 flows × 5 versions); unit = 100 bytes per 5 minutes	about 170 MB
`raw/A`	routing matrix lines: link index, flow index, fraction	54 × 144
`raw/links`	54 link names and types (0 backbone, 1 inbound, 2 outbound)	–
`raw/topo-2003-04-10.txt`	link capacities in kbit/s (9,920,000 or 2,480,000)	–
`raw_geant/totem/traffic-matrices/IntraTM-*.xml`	GÉANT, one XML file per 15 minutes, kbit/s, only listed pairs	10,772 files, ~227 MB
`raw_geant/totem/topology-anonymised.xml`	GÉANT links (no weights or capacities)	74 directed links

4.3 Processing, step by step (Abilene, Phase 1, 01_data.py)

1. **Load.** `load_abilene_raw()` reads each file with np.loadtxt, checks the shape is (2016, 720), reshapes to (2016, 144, 5) and keeps column 0 (**realOD**, the measured traffic). The 24 weeks are stacked into (24, 2016, 144) and cached as `processed/abilene_col0.npy` because parsing is slow.
2. **Confirm the column.** The script compares realOD with a model estimate (generalTomogravity) to show that the estimate follows but drifts from the measurement (figure 01_column_check.png).

3. **Clean** with `abilene_clean()` (rules R1–R4, chapter 5.2): 7,529 of 48,384 steps are removed, so **84.4% are kept**.
4. **Segment.** Kept steps are cut wherever a step was removed or two weekly files are not back to back. Result: **9 segments** of 2.3 to 46 days.
5. **Window** inside each segment (`make_windows`): X = 12 consecutive steps, y = the step h later. The timeline index of each target is kept.
6. **Split** chronologically 60/20/20 (`chronological_split`): 24,448 / 8,149 / 8,150 samples.
7. **Scale**: fit `FlowScaler` on X_train only, transform all six arrays, save everything.
8. **Exit check**: shapes and dtypes are as contracted, X_train is exactly in [0, 1], and undoing the scaling on y_test gives back the raw values (assert).

4.4 Model training and prediction (Phases 3–5)

1. Load the six arrays and the scaler.
2. Compute persistence on validation and test, the reference for this horizon.
3. For each model configuration and seed: `fit_or_load` either loads the saved model or trains it (flowchart in chapter 5.6).
4. Predict with `predict(model, X, scaler=...)`, which clips forecasts to the physical range: no negative traffic, nothing above the fastest link.
5. Undo the scaling on forecasts and truth, compute MAE/RMSE/MAPE, append a row to metrics.csv with `log_result`, print a comparison and draw figures.

4.5 Link loads and warnings (Phase 6, 04_analysis.py)

1. For each horizon (1, 6, 12), load the test inputs and the three saved CNN-LSTM seeds; average the three forecasts in original units.
2. Convert flows to backbone-link utilisation: kbit/s = ($A_{backbone}$ @ flows) × 100 × 8 / 300 / 1000; divide by capacity.
3. Do the same for the true values and for persistence.
4. For thresholds 40%, 50% (primary) and 60%: mark true events (true ≥ threshold) and warnings (forecast ≥ threshold); compute precision, recall and F1 (`warn_prf`) and onsets warned in advance (`warn_onsets`).
5. Diagnose which links and flows cause the events (one flow, LOSA→CHIN, explains 1,048 of 1,091 congested link-steps).

4.6 What happens with errors, unexpected input and exceptional cases

Situation	What the code does	Where
A raw file has the wrong shape	Raises ValueError naming the file and the expected shape	load_abilene_week
A raw week file is missing	Raises FileNotFoundError listing the missing weeks	load_abilene_raw
NaN or negative values reach the scaler	Raises ValueError ('apply the missing-data policy first')	FlowScaler._check
A flow is constant in training (max = min)	Range set to 1 to avoid division by zero	FlowScaler.range_
Scaler used before fitting	Raises RuntimeError	FlowScaler._require_fit
Processed data missing	FileNotFoundError: 'run scripts/01_data.py first'	load_splits
Segment shorter than window + horizon	Skipped (no samples made from it)	_prepare_segments

Situation	What the code does	Where
Test input above the training range	The network may extrapolate; `predict(scaler=)` clips the forecast to 0 … fastest link (Abilene 9.92 Gbit/s, GÉANT 10 Gbit/s)	predict
Prediction/truth shape mismatch or NaN	evaluate raises ValueError	evaluate
All true values below the MAPE floor	MAPE returned as NaN (coverage 0)	evaluate
Script crashes midway	Its report part is still saved, marked in red with the traceback	Report._on_error
Same model already trained	Loaded from disk instead of retrained (set RETRAIN = True to force)	fit_or_load
Onset right after a data gap	Not counted, because the previous step is unknown	warn_onsets
Precision with zero warnings	Returned as NaN, not divided by zero	warn_prf

4.7 What the user finally sees

- `results/report.html` opens in the browser: a table of every logged result (latest per distinct run), then one section per script with its printed output and figures, a timestamp, the run time and a 'Finished' or 'Stopped with an error' badge.
- `results/metrics.csv`: the permanent log of every run (raw units).
- `results/figures/*.png`: 21 figures. `models/`: every trained model, reusable without retraining.

5. Complete Logic Explanation

Each operation below is explained as: **what** it does, **why** it is needed, the **rules** that control it, its **inputs → processing → output**, the **scenarios** it handles, and **why this approach** was chosen, where the project's evidence supports that.

5.1 Choosing the measured column (realOD)

Aspect	Explanation
What	Each Abilene row has 5 numbers per flow. Only column 0 (realOD) is kept (`ABILENE_COLUMN = 0`).
Why	Columns 1–4 are gravity and tomogravity *estimates*: mathematical guesses of flows built from each router's totals. Training on them would teach the model a formula, not real traffic.
Evidence	The README lists realOD first. All five columns have exactly the same per-router totals (as the estimation methods require), so only one can be the measurement. realOD has more exact zeros, as expected from 1-in-100 packet sampling, and the estimates drift from it (01_column_check.png).

5.2 The missing-data policy (cleaning)

What: find measurement failures and remove those time steps. **Why:** training on or scoring against values that were never really measured would be dishonest; a fake zero outage looks like a sudden traffic collapse the model could never predict. **Rule of the policy:** *remove, never invent*. Gaps are not filled with guessed values.

Abilene decision table (abilene_clean, common.py:377)

Rule	Condition (all thresholds are constants in common.py)	Action	Removed
R1 Duplicate	the first 288 steps (1 day) of week w are exactly equal to the last 288 of week w−1	remove the copy; force a segment break	1,152 steps (4 days)
R2 Flow outage	a flow that is normally active (zero less than 50% of the time) reads 0 for ≥ 288 consecutive steps (1 day)	remove those steps for all flows	6,336 (22 days)
R3 Router outage	the sum of all traffic into, or out of, one router is 0 for ≥ 12 steps (1 hour)	remove those steps	40
R4 Impossible	any flow above `MAX_FLOW_RAW` = 9.92 Gbit/s for 5 minutes (3.72×10^9 raw units)	remove that step	1 (a 142 Tbit/s value)
Everything else	ordinary zeros on small flows; large but physically possible bursts	**keep**	–

Scenarios. (a) A small flow such as SNVA→ATLA-M5 is zero 86% of the time. It is not 'normally active', so its long zero runs are its normal state and do *not* trigger R2. (b) A busy flow at zero for 6 hours: below the 1-day R2 limit, kept. (c) A 1.5-hour gap of all traffic out of KSCY: R3 removes it. (d) The 6.5 Gbit/s LOSA→CHIN bursts: possible on a 9.92 Gbit/s link, so **kept**, which matters later.

```
common.py:430 (simplified): from kept steps to gap-free segments
```

```
keep = reason == ""                       # a step survives only if no rule fired
for w in range(1, n_w):                   # week joins that are not proven back to back
    joined = (starts[w] - starts[w - 1]).days == 7 and (w + 1) not in dup_weeks
    if not joined:
        new_segment[w * T] = True         # force a cut at the start of week w
segments = []
for a, b in _true_runs(keep):            # each run of kept steps...
    cuts = [a] + [i for i in where(new_segment[a+1:b]) + a + 1] + [b]
    for s, e in zip(cuts[:-1], cuts[1:]):  # ...is cut again at forced breaks
        segments.append((s, full[s:e]))
```

Why remove rather than fill: filling (interpolation) would invent values; removing costs only 15.6% of the data. **Why segments:** two steps on either side of a gap are not 5 minutes apart, so a window spanning the gap would teach the model a false 'jump'.

GÉANT decision table (geant_clean, common.py:544)

Rule	Condition	Removed slots
G1 no_file	no XML file for that 15-minute slot (8 gaps, the longest 4.75 days)	624
G2 incomplete	more than 50% of the flows that ever appear are absent (normal is 5–20%)	4
G3 impossible	any flow above 10 Gbit/s (`GEANT_MAX_FLOW_KBPS = 1e7`); values up to 9.2 Tbit/s seen	9
G4 outage	network total below 20% of the median total	4
Absent pair in a kept file	set to **0**: absent pairs are mostly tiny flows, and no listed value is ever exactly 0, so absence is read as 'below the measurement floor' (an assumption, stated as such)	–

Result: 94.4% of 11,396 slots kept, in 13 segments (longest 36.7 days).

5.3 Windowing

Aspect	Explanation
Input	one segment, array (T, F)
Rule	X[i] = series[i : i + WINDOW]; y[i] = series[i + WINDOW + HORIZON − 1]
Output	X (n, 12, F), y (n, F), with n = T − WINDOW − HORIZON + 1; plus the timeline index of every target
Scenarios	a segment shorter than WINDOW + HORIZON gives no samples and is skipped; samples from different segments are concatenated in time order
Why 12 steps	one hour of history, the plan's choice. Window length was **not tuned** (it changes the data; left open).
Implementation	fully vectorised: an index matrix idx[i, j] = i + j picks all windows at once (no Python loop)

5.4 Chronological split

```
common.py:184 chronological_split (simplified)

n_train = int(n * 0.6); n_val = int(n * 0.2)
return X[:n_train], y[:n_train], X[n_train:n_train+n_val], ..., X[n_train+n_val:], y[n_train+n_val:]
```

Why: a random split would leak the future (neighbouring windows share 11 of 12 steps). **Side effect:** the test period (mid-Aug to early Sep 2004) is burstier than validation, which explains why validation and test rankings sometimes disagree. This is a real-world effect, and it is reported rather than hidden.

5.5 Scaling and its inverse

Step	Formula	Why
fit (train inputs only)	$\min_k$, $\max_k$ of log(1 + x) for each flow k	no future information
transform	$z = (\log(1 + x) - \min_k) / \text{range}_k$, range = max − min (or 1 if 0)	0..1 per flow
inverse	$x = \exp(z \cdot \text{range}_k + \min_k) - 1$	metrics in real units
checks	NaN or negative input → error	catch cleaning mistakes early

Scenario: values outside 0..1. A very small share of validation and test values fall outside [0, 1] (the log reports 0.02% below and at most 0.26% above), but some of them are far outside: one validation input reaches 1.34, and LOSA→CHIN test inputs reach 1.28 at horizon 6. They are deliberately **not clipped** in the data. Clipping would hide real behaviour. Instead the *forecasts* are clipped to the physical range (5.8).

5.6 Building the model: plain vs residual

Total 143,608 trainable parameters (Abilene); 237,163 on GÉANT (529 flows).

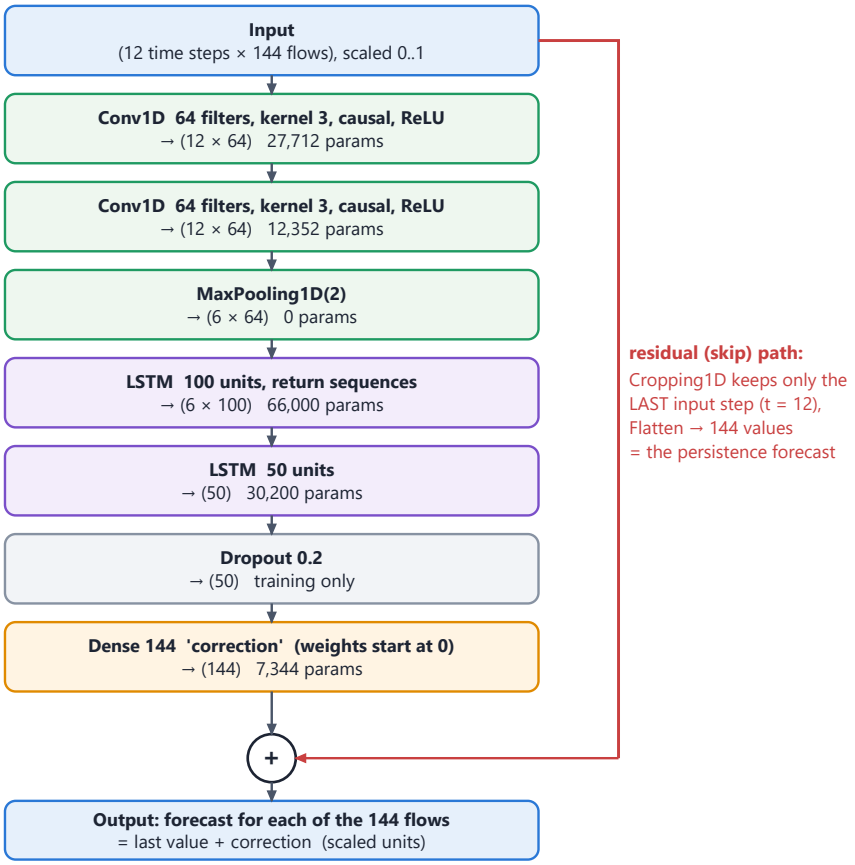


Diagram: *The final residual CNN-LSTM with real output shapes and parameter counts (Keras summary).*

Decision logic in `_output()`: if residual is False, the network ends in Dense(144) and outputs the level directly (the original plan). If residual is True, a Dense(144) layer named 'correction', with **kernel initialised to zeros**, is added to the last input step. Its bias also starts at 0 (the Keras default), so before training the output equals the persistence forecast exactly. The project verified this in code.

Why this was chosen: the evidence in section 13 of the log. Plain LSTM-only had 9.08 Mbit/s test MAE (2.8× persistence) and beat persistence on only 2 of 144 flows; the residual LSTM-only reached 3.23 Mbit/s and

won on 48 flows. Both versions are kept in the record so the comparison is itself a finding.

5.7 Training: loss choice, optimiser and stopping rules

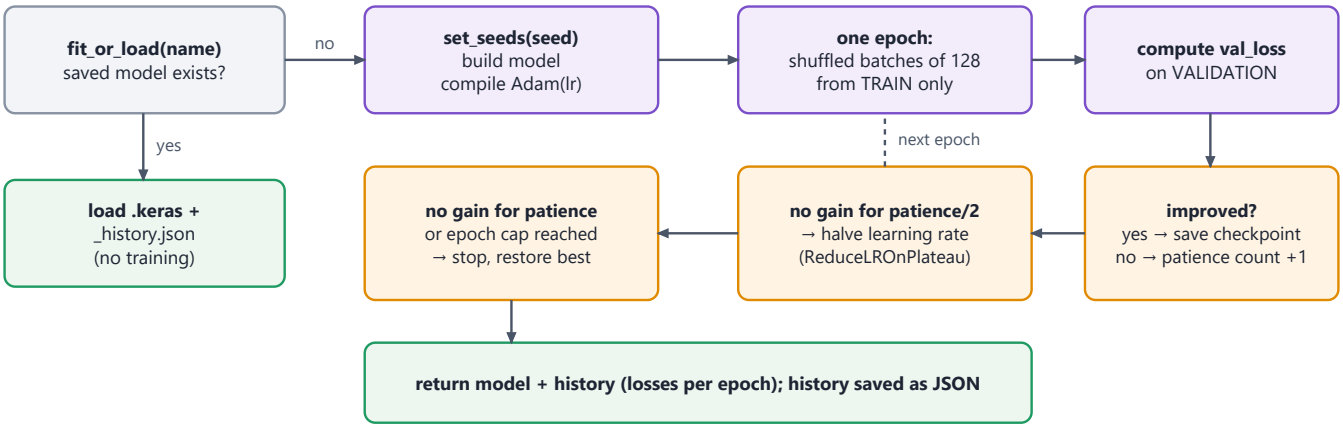


Diagram: *Training control flow (`fit_or_load` and `train_model`, common.py:790–851).*

The loss function was the decisive design choice

Loss (training objective)	What it measures	Residual CNN-LSTM, validation MAE (persistence 2.742)
mse	squared error on scaled (log) values; every flow counts equally	2.777 (after removing one blown-up example) → worse
mae	absolute error on scaled values	2.772 → worse
mae_raw	absolute error in **original units**: undo min–max and log inside the graph, then |true − pred|, divided by the mean training traffic	**2.693 → better** (LSTM-only 2.646)

```
common.py:783 make_loss('mae_raw')

def mae_raw(y_true, y_pred):
    back = lambda z: tf.math.expm1(z * rng + lo)          # scaled -> raw units, inside TensorFlow
    return tf.reduce_mean(tf.abs(back(y_true) - back(y_pred))) / ref   # ref = mean training traffic
```

Why it works (the project's explanation, raised as a hypothesis in log section 13.5 and confirmed by the loss comparison in 14.1): on a log scale a tiny flow and a huge flow count equally, so the model spent its effort on flows that hardly matter in Mbit/s. The reported metric (MAE in Mbit/s) is dominated by big flows. Training on exactly the metric you are judged on aligns the two. Dividing by the mean traffic only keeps the loss near 1 for numerical comfort; it does not change the optimum.

Training setting	Value	Reason
Optimiser	Adam, learning rate 0.001	standard, robust default; tuning 3e-4 and 3e-3 did not help
Batch size	128 samples, shuffled each epoch	shuffling within the training split cannot leak across splits
Epoch cap	100 (Phase 3), 200 (Phase 4+)	best 03b model was still improving at 100
EarlyStopping	patience 10 (later 15) epochs; restore best weights	stop when validation stops improving; avoid overfitting
ReduceLROnPlateau	halve LR after patience // 2 epochs; min 1e-5	finer steps when progress stalls
ModelCheckpoint	save the best epoch to models/*name*.keras	the saved model is the best on validation

5.8 Prediction with a physical cap

What: `predict(model, X, scaler=sc)` clips forecasts in scaled units to the range that corresponds to 0 … the fastest link. **Why (a real bug that was found):** one validation example had an input at 1.34 (above the training maximum of 1.0); the MSE-trained model extrapolated to 1.99, and the inverse transform (an exponential) turned that into about 25 million Mbit/s, making validation MAE 23.99 instead of 2.78. **Rule:** lo = scaled(0), hi = scaled(MAX_FLOW_RAW) per flow; GÉANT passes max_raw = 10 Gbit/s.

> **NOTE Where the cap is applied**
> All Phase 4+ numbers (03c, 03d, 03e, 04, 05) use the cap. The Phase 3 and 03b scripts call predict without a scaler (no cap); the blown-up 03b number is reported in the log together with its explanation.

5.9 Evaluation logic

1. Convert forecasts and truth back to raw units (`evaluate_scaled` calls inverse_transform on both).
2. Check: same shape, no NaN (else ValueError).
3. MAE = mean |pred − true|; RMSE = sqrt(mean (pred − true)^2), over all samples and flows.
4. MAPE only where true ≥ 10% of the mean true value; coverage = share of entries included (about 74% on Abilene test).
5. Report the % change against persistence on the *same* split and horizon. Positive = better.
6. Log to metrics.csv (raw units; the page converts Abilene to Mbit/s with `MBIT`).

5.10 ARIMA procedure (02_baselines.py)

1. Pick the 5 busiest flows by mean training traffic.
2. Fitting data: the last 4 weeks of the unbroken segment where training ends (ARIMA needs an unbroken series), on log1p values.
3. For each of 7 orders ((1,0,0), (2,0,0), (1,0,1), (2,0,1) with a constant; (0,1,1), (1,1,1), (2,1,1) without), fit and forecast one step ahead over the validation targets. Keep the order with the lowest validation MAE.
4. Forecast the test targets with the parameters fixed, running the model separately inside each segment so it never crosses a gap.
5. Score on exactly the same flows and test samples as persistence (an assert checks the truth arrays are identical).

A bias correction (log-normal mean instead of median) was tried and made MAE worse, so it is not used.

5.11 Tuning rule and seeds (03c_tuning.py)

```
the one-factor-at-a-time rule (simplified)

cfg = {"filters": 64, "kernel": 3, "units": (100, 50), "dropout": 0.2, "lr": 1e-3}
for key, options in GRID:                    # one setting at a time
    for opt in options:
        v = validation_MAE(train({**cfg, key: opt}))
        if v < best * (1 - 0.005):           # must beat the current best by more than 0.5%
            keep opt
```

Outcome: no option beat the base by more than 0.5%, so the planned settings stayed. **Why the 0.5% margin:** the seed-to-seed spread is about 0.03 Mbit/s (about 1%), so smaller differences are noise. **Why one at a time:** 10 trials instead of a full grid of 3^5 = 243. The trade-off is that interactions between settings are not explored.

5.12 Horizons and ablations (03d_horizons.py)

- Each horizon gets its own windows, split and scaler (`processed/h6`, `processed/h12`) and its own persistence reference, so MAE is comparable only within a horizon.
- Main comparison: CNN-LSTM, LSTM-only, CNN-only, 3 seeds each. Horizon 1 reuses the Phase 4 models when the settings are identical.
- Ablations (1 seed): 1 conv layer (`n_conv=1`), no pooling (`pool=False`), 1 LSTM layer (`units=(100,)`).
- There is no selection step in this script, so validation and test are both reported.

5.13 Congestion-warning logic (04_analysis.py)

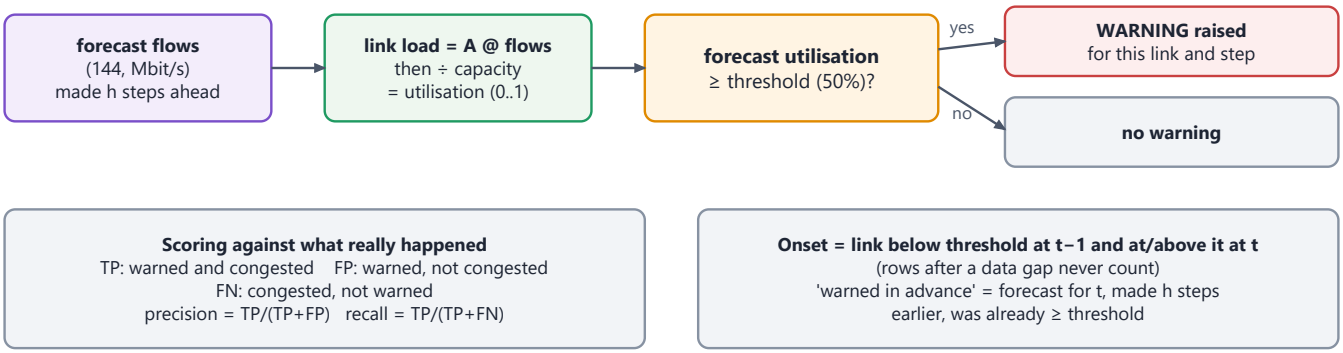


Diagram: *From forecast flows to scored warnings.*

```
common.py:641

def warn_onsets(true_ev, steps):
    consecutive = np.r_[False, np.diff(steps) == 1]          # row follows the previous row in time?
    prev = np.vstack([np.zeros((1, n_links), bool), true_ev[:-1]])
    return true_ev & ~prev & consecutive[:, None]             # congested now, not before, no gap
```

Case at a link and step	true event	warning	Counts as
load really ≥ 50% and forecast ≥ 50%	yes	yes	TP (true positive)
forecast ≥ 50% but load < 50%	no	yes	FP (false alarm)
load ≥ 50% but forecast < 50%	yes	no	FN (missed)
both below	no	no	TN (not used by P/R/F1)

> **KEY POINT A consequence of the definitions you should be able to explain**
>
> At horizon 1, **persistence can never warn of an onset in advance**: an onset at step t means the load at t−1 was below the threshold, and the persistence forecast for t is exactly the load at t−1. The residual CNN-LSTM is persistence plus a tiny correction, so it also warned 0 of 35 onsets. At horizons 6 and 12 'warnings in advance' can happen only if the load 6 or 12 steps earlier was already high, for example from an earlier episode that briefly dipped. That is why both methods warn the same few onsets (5 of 35 at 30 minutes).

Threshold logic: the plan's 80% gives zero events (busiest link 75%), so 50% was fixed before looking at warning results, with 40% and 60% as a sensitivity check. This deviation is documented.

5.14 Approximate routing for GÉANT

Problem: the anonymised topology has links but no routing weights or capacities. **Logic:** (1) build the adjacency matrix of the 74 directed links; (2) all-pairs hop distances with Floyd–Warshall; (3) count shortest paths from each source; (4) a link (u, v) lies on a shortest s→t path when dist(s,u) + 1 + dist(v,t) = dist(s,t), and the share of the flow on it is paths(s,u) · paths(v,t) / paths(s,t), which is equal splitting over all shortest paths. **Consequence:** 'high load' on GÉANT means the top 10% of a link's own training loads, **not** congestion against capacity. Its link-level results are approximate.

5.15 Error-analysis definitions (05_errors.py)

Quantity	Definition (fixed before looking at test results)
move	|true − last observed value| / the flow's mean training traffic
move bins	percentiles of the pooled test moves: smallest 50%, 50–80, 80–95, 95–99, top 1%
damping	slope through the origin of (forecast − last) on (true − last): Σ a·b / Σ b^2; 1 = move fully forecast, 0 = persistence
peak	a target above the flow's own 95th percentile of training targets
peak bias	Σ (forecast − true) / Σ true over peaks, in %
forecast used	mean of the 3 seeds' forecasts in original units (slightly better than a single seed)

5.16 Logging and the results page

- Every result is appended to metrics.csv. Rows are never overwritten, so re-runs create duplicates by design; the page shows the latest row per (name, dataset, horizon, notes).
- Each script's printout and figures are saved as `results/reports/<script>.json`; the page is rebuilt from all parts, so running one script updates only its own section.
- Figures are embedded in the page as base64 PNGs, so `report.html` is self-contained (2.8 MB).

6. Source Code Explanation

6.1 Folder structure

```
traffic_project/
├── common.py              the shared library (1,200 lines): everything reusable lives here
├── requirements.txt       numpy, pandas, matplotlib, statsmodels, tensorflow, ipykernel
├── README.md              setup, layout, data contract, rules
├── .gitignore             excludes .venv, raw/, processed/, models/, report pages
├── .vscode/settings.json  points VS Code at .venv
├── docs/
│   ├── PROJECT_LOG.md     plain-language record of every phase (sections 1-18)
│   └── presentation/      10-slide mid-project review deck (8 Oct)
├── scripts/               00_setup … 05_errors, one per phase, cells marked '# %%'
├── raw/                   Abilene X01..X24.gz, A, links, demands, topo, readme (not in git)
├── raw_geant/             GÉANT TOTEM matrices and topology (should be git-ignored)
├── processed/             .npy splits, scaler.npz, target_steps.npy; h6/, h12/, geant/h1, geant/h4
├── models/                ~100 .keras models + _history.json
└── results/               metrics.csv, figures/, reports/, *_run.log, report.html
```

6.2 How a script is organised

```
the header every script shares (05_errors.py shown)

# %% header: locate common.py, import it, seed, show status
_p = Path(globals().get("__file__", Path.cwd())).resolve()
while not (_p / "common.py").exists():   # walk up the folders until common.py is found
    _p = _p.parent
sys.path.insert(0, str(_p))
import numpy as np, tensorflow as tf
import common; importlib.reload(common)  # pick up edits without restarting VS Code
from common import *
report = Report("05_errors", "Phase 7: Error analysis")   # start capturing output
set_seeds(); status()
# %% 1. ...   each '# %%' line starts a cell that can be run on its own in VS Code
...
report.finish()                          # save this script's part, rebuild report.html
```

Why '# %%' cells instead of notebooks: the same run-a-cell workflow in VS Code, but the files are plain text: easy to diff in git and runnable from a terminal. **Why reload:** in an Interactive Window Python keeps modules cached; reloading makes edits to common.py take effect immediately.

6.3 The important functions, in the required format

FlowScaler (common.py:96)

1. Purpose	Scale traffic to [0, 1] per flow with a log, and undo it exactly.
2. Inputs	fit(a): any array whose last axis is flows (fit on X_train). transform / inverse_transform: arrays of the same layout.
3. Internal logic	fit: log1p, reshape to (−1, F), per-column min and max. transform: (log1p(a) − min) / range. inverse: expm1(z·range + min). save/load: np.savez with min_ and max_.
4. Conditions / edge cases	NaN or negative input → ValueError; unfitted → RuntimeError; constant flow → range 1.
5. Output / side effects	float32 arrays; scaler.npz on disk.
6. Used by	_prepare_segments (fit + save), every model script (load), make_loss (min/range constants), predict (cap).

7. Worked example	WASH→WASH: min 0, max 19.845. x = 125,594,976 → log1p = 18.649 → z = 0.9397. Inverse of 0.9397 gives back 125,594,976.

make_windows (common.py:164)

1. Purpose	Turn a (T, F) series into supervised samples.
2. Inputs	series (T, F); window = 12; horizon = 1, 6 or 12.
3. Internal logic	n = T − window − horizon + 1; idx = arange(window)[None,:] + arange(n)[:,None]; X = series[idx]; y = series[window+horizon−1 : window+horizon−1+n].
4. Conditions / edge cases	Not 2-D → ValueError; n ≤ 0 → ValueError ('series too short').
5. Output / side effects	X (n, window, F), y (n, F), float32.
6. Used by	_prepare_segments, self_test.
7. Worked example	A segment of 288 steps (1 day), window 12, horizon 6 → n = 288 − 12 − 6 + 1 = 271 samples; y[0] is step 17 (0-based).

abilene_clean (common.py:377)

1. Purpose	Apply rules R1–R4 and cut the kept data into gap-free segments.
2. Inputs	weeks: (24, 2016, 144) raw.
3. Internal logic	1) R1 compare each week's first day with the previous week's last day. 2) R2 for active flows find zero runs ≥ 288 using _true_runs. 3) R3 reshape to (T, 12, 12), sum each router's column/row, find zero runs ≥ 12. 4) R4 any value > MAX_FLOW_RAW. A 'reason' string per step records the first rule that fired. 5) Segment breaks at removed steps and at non-adjacent week joins.
4. Conditions / edge cases	A step can match several rules; only the first reason is stored (np.where(reason == '', ...)). Small, mostly-zero flows are exempt from R2.
5. Output / side effects	(segments, info): list of (start index, array) and a dict with keep mask, reasons and counts.
6. Used by	prepare_abilene, 01_data (figures), 02_baselines (ARIMA needs the raw segments).
7. Worked example	The 22-day ATLA-M5 outage: 11 inbound flows read 0 from the last day of X02 to the end of X05. R2 fires → 6,336 steps removed.

_prepare_segments (common.py:463)

1. Purpose	The shared back half of the data pipeline for both networks.
2. Inputs	segments [(start, array)], info, window, horizon, output directory.
3. Internal logic	Window each segment, record target index start + window + horizon − 1 + i, concatenate, split 60/20/20, fit scaler on X_train, transform all six arrays, save arrays + scaler + target_steps.
4. Conditions / edge cases	Short segments skipped. Prepare_abilene was refactored onto this function and its output was checked identical.
5. Output / side effects	Files in processed/...; returns a summary (counts, where train/val end).
6. Used by	prepare_abilene, prepare_geant (03d and 03e create h6, h12, geant/h1, geant/h4 when missing).
7. Worked example	Abilene h=1: 40,747 samples → 24,448 / 8,149 / 8,150.

_output and build_cnn_lstm (common.py:703, 746)

1. Purpose	Build the network; optionally residual.
2. Inputs	window, n_flows (144 or 529), filters, kernel, units, dropout, residual, n_conv, pool.

3. Internal logic	Input → n_conv × Conv1D(causal, ReLU) → optional MaxPooling1D(2) → LSTM layers (all but the last return sequences) → Dropout → _output. _output: plain Dense(n_flows), or Dense(n_flows, zeros) + Cropping1D((window−1, 0)) + Flatten + Add.
4. Conditions / edge cases	TensorFlow imported inside the function so non-model scripts start fast.
5. Output / side effects	A Keras Model named cnn_lstm or cnn_lstm_res.
6. Used by	03b, 03c, 03d, 03e (with n_flows = 529); ablations via n_conv, pool, units.
7. Worked example	Abilene residual: 143,608 parameters; the same count as the plain model (the skip path has no weights).

train_model and fit_or_load (common.py:790, 829)

1. Purpose	Train with good defaults, or reuse a saved model.
2. Inputs	model builder, name, train/val arrays, retrain flag, seed, epochs, batch, lr, patience, loss.
3. Internal logic	fit_or_load: if models/name.keras and its history exist and retrain is False → load (compile=False). Else set_seeds(seed), build, train_model, save history JSON. train_model: compile Adam(lr) with the loss; callbacks ModelCheckpoint, EarlyStopping(restore_best_weights), ReduceLROnPlateau, progress printer; model.fit(shuffle=True).
4. Conditions / edge cases	Changing a model's code without RETRAIN = True would silently reuse the old file; the docstring warns about this.
5. Output / side effects	(model, history dict, trained flag); files models/name.keras, name_history.json.
6. Used by	Every model script.
7. Worked example	p4_cnn_lstm_..._s44: best epoch 73 of 88 (from metrics.csv notes).

predict (common.py:854)

1. Purpose	Get forecasts, clipped to what is physically possible.
2. Inputs	model, X (scaled), scaler, max_raw.
3. Internal logic	model.predict in batches of 1024; if a scaler is given: lo = scaled(0), hi = scaled(max_raw) per flow; np.clip.
4. Conditions / edge cases	Without a scaler there is no clipping (Phase 3 / 03b).
5. Output / side effects	Scaled forecasts (n, F).
6. Used by	All evaluation and analysis scripts.
7. Worked example	For WASH→WASH the cap in scaled units is log1p(3.72e9)/19.845 = 1.110.

evaluate (common.py:217)

1. Purpose	Compute metrics in real units.
2. Inputs	y_true, y_pred (raw units), optional MAPE floor.
3. Internal logic	Check shapes and NaN; err = pred − true; MAE, RMSE; floor = 0.1 × mean(true); MAPE over true ≥ floor; coverage = share included.
4. Conditions / edge cases	No value above the floor → MAPE = NaN.
5. Output / side effects	dict: mae, rmse, mape, mape_coverage.
6. Used by	evaluate_scaled, all scripts; logged by log_result.
7. Worked example	Chapter 7.4 works through 4 numbers by hand.

warn_prf and warn_onsets (common.py:632, 641)

1. Purpose	Score warnings; find congestion onsets.
2. Inputs	Boolean arrays (rows, links) of true events and warnings; the timeline index per row.
3. Internal logic	TP/FP/FN by logical AND/NOT; precision, recall, F1 with zero-division guards. Onset = event now, not on the previous row, and the previous row is the previous time step.
4. Conditions / edge cases	No warnings → precision NaN; no TP → F1 0 (or NaN if no events at all).
5. Output / side effects	(p, r, f, tp, fp, fn); boolean onset array.
6. Used by	04_analysis (Abilene), 03e (GÉANT).
7. Worked example	Chapter 9, example 2.

load_geant_raw and load_geant_routing (common.py:501, 589)

1. Purpose	Read 10,772 XML matrices; build approximate routing.
2. Inputs	raw_geant/ files.
3. Internal logic	Regex-parse each file's date and its src/dst values into a 23 × 23 matrix; place it in row (time − 1 Jan 2005) / 15 min; unlisted pairs NaN; cache. Routing: adjacency → Floyd–Warshall → shortest-path counts → fractional A (74 × 529).
4. Conditions / edge cases	Missing files leave NaN rows (handled by geant_clean G1).
5. Output / side effects	(11,396, 529) float32 array; dict with A, link names, hop distances.
6. Used by	prepare_geant, 03e_geant.
7. Worked example	Flow index = (src − 1) × 23 + (dst − 1).

Report and build_results_page (common.py:892, 986)

1. Purpose	Collect each script's output into one HTML results page.
2. Inputs	Script name and title; whatever the script prints; figures via show().
3. Internal logic	Tee stdout; blocks of text/heading/image/error; on finish write JSON; rebuild the page from all JSON parts plus a summary of metrics.csv (latest per distinct run); open the browser.
4. Conditions / edge cases	Re-running in the same session restores stdout first; errors are captured by sys.excepthook; atexit saves on normal exit.
5. Output / side effects	results/reports/name.json, results/report.html.
6. Used by	Every script.
7. Worked example	Running 04_analysis.py updates only the 'Phase 6' section of the page.

self_test (common.py:1143)

1. Purpose	Prove the data pipeline works before using real data.
2. Inputs	None (makes 2 weeks of synthetic traffic with a daily sine pattern, log-normal flow sizes, an always-zero flow and a zero stretch).
3. Internal logic	Asserts on window shapes and alignment, split order and completeness, scaler range and round trip, save/load identity, persistence evaluation (MAE > 0; perfect forecast MAE = 0) and logging, all in a temporary folder.
4. Conditions / edge cases	Touches no project files.
5. Output / side effects	Prints 'self_test passed …' or raises AssertionError.
6. Used by	00_setup.py; python common.py.

7. Worked example	–

6.4 Tracing one complete run through the code

#	Call	Effect
1	01_data: load_abilene_raw()	24 files → (24, 2016, 144), cached
2	abilene_clean(weeks)	reasons per step → 9 segments
3	prepare_abilene() → _prepare_segments()	make_windows, chronological_split, FlowScaler.fit/transform, save
4	03c: load_splits(), FlowScaler.load()	same data for every experiment
5	make_loss('mae_raw', scaler, REF)	a TensorFlow loss in original units
6	fit_or_load(build_cnn_lstm(residual=True), 'p4_cnn_lstm_…_s42', …)	train (or load) and checkpoint
7	predict(model, X_test, scaler=scaler)	scaled, clipped forecasts
8	evaluate_scaled(y_test, p, scaler)	MAE/RMSE/MAPE in raw units
9	log_result(name, m, notes=…)	row appended to metrics.csv
10	04_analysis: utilisation(flows)	A_bb @ flows → kbit/s → ÷ capacity
11	warn_prf / warn_onsets	precision, recall, F1, onsets
12	report.finish()	reports/04_analysis.json → report.html

6.5 Non-obvious statements worth knowing

Statement	Why it is there
`layers.Cropping1D((window - 1, 0))(inp)`	removes the first 11 steps, leaving only the last one: the persistence value
`kernel_initializer="zeros"`	makes the untrained residual model exactly persistence
`padding="causal"`	no Conv1D output can see a later input step
`tf.config.experimental.enable_op_determinism()`	same seed → identical results on every run
`load_model(path, compile=False)`	saved models are only used to predict, so the custom loss need not be rebuilt
`np.where(r > 0, r, 1.0)`	avoids division by zero for a constant flow
`MAX_FLOW_RAW = 9_920_000 * 1e3 / 8 * 300 / 100`	9.92 Gbit/s link → bytes per 5 min → units of 100 bytes
`MBIT = 100 * 8 / 300 / 1e6`	raw unit → Mbit/s averaged over the 5 minutes
`LOSA_CHIN = 7 * 12 + 2`	flow index from router order (LOSA is 7th, CHIN 2nd, counting from 0)
`consecutive = np.r_[False, np.diff(steps) == 1]`	an onset just after a data gap is not trusted

7. Algorithms and Mathematical Logic

For each formula: the intuition, the variables, the steps, a numerical example and its effect on the project. Section 7.16 separates what is **implemented** from what is only **discussed**.

7.1 Unit conversion: raw units → Mbit/s

Intuition: the dataset stores 'hundreds of bytes per 5 minutes'; people think in megabits per second. **Formula:** Mbit/s = raw × 100 bytes × 8 bits / 300 s / 10^6 = raw × 2.667 × 10^{-6} (`MBIT` in common.py).

Step	Value (WASH→WASH, last input of example 1)
raw value	125,594,976 (units of 100 bytes in 5 min)
× 100 → bytes	12,559,497,600 bytes in 300 s
× 8 → bits	100,475,980,800 bits
÷ 300 s	334,919,936 bit/s
÷ 10^6	**334.9 Mbit/s**

The same idea gives the cleaning limit: a 9.92 Gbit/s link carries at most 9.92 × 10^9 / 8 × 300 / 100 = **3.72 × 10^9** raw units in 5 minutes (`MAX_FLOW_RAW`).

7.2 Scaling: log1p + per-flow min–max

Variables: x = raw value; k = flow; m_k, M_k = min and max of log(1+x) for flow k over the training inputs. **Forward:** $z = (\log(1 + x) - m_k) / (M_k - m_k)$. **Inverse:** $x = \exp(z\,(M_k - m_k) + m_k) - 1$.

Quantity (WASH→WASH)	Value
m (training min of log1p)	0.000 (the flow has at least one zero in training)
M (training max of log1p)	19.845
x = 125,594,976 → log(1 + x)	18.6486
z = 18.6486 / 19.845	**0.9397**
inverse of 0.9360 (a forecast)	exp(0.9360 × 19.845) − 1 ≈ 116.7 million raw = 311 Mbit/s

> **CAUTION Why small scaled errors are big real errors**
>
> Near the top of the range, a change of 0.004 in z is a change of 0.004 × 19.845 = 0.079 in log, a factor $e^{0.079}$ = 1.08, which is 8% of the traffic. That is why the project never reports scaled errors, and why training on scaled MSE favoured the wrong flows.

7.3 Window count

For a segment of length T, window W and horizon h: **n = T − W − h + 1**. Example: a 46-day segment has 46 × 288 = 13,248 steps; at W = 12, h = 1 that gives 13,236 samples, and at h = 12, 13,225. Summed over the 9 Abilene segments at h = 1 the total is 40,747 samples.

7.4 MAE, RMSE, MAPE and coverage, worked by hand

Illustrative numbers (not project data): true = [100, 50, 2, 0], forecast = [90, 55, 4, 1].

Metric	Formula	Calculation	Result
error e	forecast − true	[−10, 5, 2, 1]	–
MAE	(1/n) Σ |e|	(10 + 5 + 2 + 1) / 4	**4.5**
RMSE	sqrt((1/n) Σ e^2)	sqrt((100 + 25 + 4 + 1) / 4) = sqrt(32.5)	**5.70**
MAPE floor	0.1 × mean(true)	0.1 × 38	3.8
MAPE	mean |e| / true over true ≥ floor	(10/100 + 5/50) / 2	**10%**
coverage	share of entries ≥ floor	2 of 4	**50%**

Without the floor, the entry with true = 0 would give an infinite percentage and true = 2 would give 100%: tiny flows would dominate MAPE. RMSE > MAE always signals that some errors are much bigger than others; on Abilene the persistence RMSE (17.5 Mbit/s) is 5× its MAE (3.2), because of bursts.

7.5 Improvement over persistence

gain = (1 − MAE_{model} / $MAE_{persistence}$) × 100%, positive = better. Example (Abilene test, 5 min, CNN-LSTM 3-seed mean of per-seed MAE): 1 − 3.200 / 3.212 = **+0.37% ≈ +0.4%**. With the seed spread of ±0.005 Mbit/s this is small but consistent (all 3 seeds beat persistence).

7.6 Error as a share of traffic (WAPE), computed for this guide

WAPE = Σ |forecast − true| / Σ true over all flows and test samples. Unlike MAPE it weights flows by size and never divides by zero. These values were computed for this guide from the saved models (3-seed mean forecast); they are not in the project's metrics.csv.

Network, horizon	Persistence	CNN-LSTM	R^2 (CNN-LSTM)
Abilene, 5 min	13.5%	13.4%	0.961
Abilene, 30 min	22.3%	22.1%	0.840
Abilene, 60 min	25.9%	25.5%	0.748
GÉANT, 15 min	10.9%	10.8%	0.905
GÉANT, 60 min	17.6%	17.0%	0.783

R^2 = 1 − Σ(error^2) / Σ(true − flow mean)^2. It is high largely because persistence is already good (persistence R^2 at 5 min = 0.960).

7.7 Layer arithmetic: where the 143,608 parameters come from

Layer	Formula	Calculation	Parameters
Conv1D #1	kernel × in_channels × filters + filters	3 × 144 × 64 + 64	27,712
Conv1D #2	same	3 × 64 × 64 + 64	12,352
MaxPooling1D	no weights	–	0
LSTM 100	4 × (units × (inputs + units) + units)	4 × (100 × (64 + 100) + 100)	66,000
LSTM 50	same	4 × (50 × (100 + 50) + 50)	30,200
Dense 144	inputs × outputs + outputs	50 × 144 + 144	7,344
Total			**143,608**

The '4 ×' in the LSTM is its four gates (forget, input, candidate, output), each a small Dense layer over [x(t), h(t−1)]. On GÉANT only the first conv layer and the output depend on the number of flows: 3 × 529 × 64 + 64 = 101,632 and 50 × 529 + 529 = 26,979, total **237,163**.

7.8 The LSTM equations

```
standard LSTM step (as implemented by Keras)

f(t) = sigmoid(W_f · [h(t-1), x(t)] + b_f)      forget gate     (0..1)
i(t) = sigmoid(W_i · [h(t-1), x(t)] + b_i)      input gate      (0..1)
g(t) = tanh   (W_g · [h(t-1), x(t)] + b_g)      candidate       (-1..1)
o(t) = sigmoid(W_o · [h(t-1), x(t)] + b_o)      output gate     (0..1)
c(t) = f(t) * c(t-1) + i(t) * g(t)              memory: keep some old, add some new
h(t) = o(t) * tanh(c(t))                        output passed on
```

Intuition: if the forget gate is near 1 and the input gate near 0, the memory carries information unchanged across steps, which is how an LSTM avoids the 'vanishing gradient' problem of simple recurrent networks. Here each LSTM reads only 6 steps (after pooling), so long-memory ability is barely needed. That is one reason the LSTM adds little.

7.9 Training algorithms

Algorithm	Intuition	Setting
Mini-batch gradient descent	show 128 samples, measure the loss, nudge every weight in the direction that lowers it; repeat over all 24,448 samples = 1 epoch (191 updates)	batch 128
Adam	gradient descent with a per-weight adaptive step size (running averages of the gradient and its square)	lr 0.001, β defaults
Early stopping	stop when validation loss has not improved for 'patience' epochs; restore the best weights	patience 10 / 15
Reduce LR on plateau	halve the step size when progress stalls	factor 0.5, min 1e-5
Dropout	randomly zero 20% of the 50 summary values per update; scale the rest (inverted dropout)	rate 0.2

Cost: each epoch is about 191 forward/backward passes over a 143,608-parameter network: seconds to a minute on a laptop CPU. A GPU is not required.

7.10 The original-units loss (mae_raw)

Formula: $L = \text{mean}_{\text{batch, flows}} \, | \, g(y) - g(\hat{y}) \, | \, / \, \text{ref}$, with $g(z) = \exp(z \cdot \text{range} + \text{min}) - 1$ (the inverse scaling, done inside TensorFlow so gradients flow through it) and ref = mean training traffic. **Effect:** big flows dominate, exactly as in the reported MAE. In the training history the validation loss starts at 0.1163 (≈ persistence's 2.742 / 23.5 Mbit/s mean = 0.1165) and ends at 0.1140 (= 2.68 Mbit/s): a 2% improvement.

7.11 Link utilisation

Formula: load_l (kbit/s) = Σ_k A[l, k] · flow_k (raw) × 100 × 8 / 300 / 1000; $\text{utilisation}_l = \text{load}_l / \text{capacity}_l$. **Example (measured):** at the onset in chapter 9 the DNVR→KSCY link carried 26 flows; LOSA→CHIN alone carried 4,849 Mbit/s, and the link was at 52.8% of 9.92 Gbit/s ≈ 5,238 Mbit/s.

7.12 Precision, recall, F1

Illustrative numbers: 90 TP, 5 FP, 7 FN. Precision = 90 / 95 = 0.947 (how many warnings were right). Recall = 90 / 97 = 0.928 (how many events were caught). F1 = 2PR / (P + R) = 0.937 (the harmonic mean, which punishes imbalance). **Why F1 is high on Abilene for both methods:** congestion episodes last many steps, and once a link is congested it tends to stay so. Persistence is right on every step except the first and last of each episode.

7.13 Damping (how much of a move is forecast)

Let b = true − last (the real move) and a = forecast − last (the forecast move). Least-squares slope through the origin: **$d = \Sigma ab / \Sigma b^2$**. Illustrative: b = [10, −20, 30], a = [1, −1, 2] → Σab = 10 + 20 + 60 = 90, Σb^2 = 1,400 → d = 0.064 (the model forecasts about 6% of each move). **Measured:** 0.01 (Abilene 5 min), 0.03 (30 min), 0.05 (60 min), 0.02 and 0.05 on GÉANT.

7.14 Floyd–Warshall and shortest-path counting (GÉANT routing)

```
load_geant_routing (common.py:589), simplified

dist = 1 where a link exists, INF elsewhere, 0 on the diagonal
for k in nodes:                     # allow paths through node k
    dist = min(dist, dist[:, k] + dist[k, :])   # vectorised over all (i, j)
paths[s][v] = sum of paths[s][u] over neighbours u with dist[s][u] + 1 == dist[s][v]
A[link(u,v), flow(s,t)] = paths[s][u] * paths[v][t] / paths[s][t]   if (u,v) lies on a shortest s->t path
```

Complexity: Floyd–Warshall is $O(n^3) = 23^3 \approx 12{,}000$ steps; building A loops over 529 flows × 74 links. Both are trivial at this size. **Assumption:** real GÉANT routing uses IGP weights, not hop counts, so A is an approximation.

7.15 ARIMA in one equation

For ARIMA(p, d, q) on y = log1p(traffic): (after differencing d times) $y_t = c + \varphi_1 y_{t-1} + \ldots + \varphi_p y_{t-p} + \theta_1 \varepsilon_{t-1} + \ldots + \theta_q \varepsilon_{t-q} + \varepsilon_t$. Parameters are fitted by maximum likelihood (statsmodels). Chosen orders: (2,0,1) WASH→WASH, (1,1,1) LOSA→CHIN, (2,1,1) CHIN→LOSA and WASH→NYCM, (1,0,0) NYCM→NYCM.

7.16 Implemented vs only discussed

Implemented and measured	Discussed or proposed only (not implemented)
persistence; ARIMA (7 orders, 5 flows, 1 step ahead)	seasonal ARIMA with a 1-day season (too slow to fit; noted as a limitation)
log1p + min–max scaling; chronological split; segments	seasonal naive baseline ('same time yesterday')
LSTM-only, CNN-only, CNN-LSTM; plain and residual	time-of-day / day-of-week input features
MSE, MAE and original-units MAE losses	window-length tuning; ARIMA at 30 and 60 minutes
one-at-a-time tuning; 3 seeds; ablations	real GÉANT routing and capacities (not in the data)
horizons 1/6/12 (Abilene), 1/4 (GÉANT)	acting on warnings (rerouting); online / real-time use
link utilisation, thresholds, P/R/F1, onsets	a third dataset (e.g. CESNET-TimeSeries24, Milan)
error analysis: move bins, damping, peaks, one-flow test	log-normal bias correction (tried; made MAE worse; not used)

8. Technologies, Tools and Dependencies

Technology (version used)	What it is	Role in this project	Essential?
Python 3.13.5	programming language	everything; 3.13 because TensorFlow does not support 3.14 yet	yes
venv (.venv/)	isolated Python environment	keeps this project's libraries separate; VS Code is pointed at it	replaceable (conda)
NumPy 2.5.3	array library	all data handling: windows, scaling, metrics, routing, warnings	yes
TensorFlow 2.21.0 / Keras	deep-learning framework	builds, trains, saves and runs the networks; the custom loss runs inside it	yes (PyTorch could replace it)
statsmodels 0.15.0	statistics library	ARIMA baseline	only for Phase 2
pandas 3.0.6	table library	reads metrics.csv for the summary table and comparisons	replaceable
matplotlib 3.11.2	plotting	all 21 figures, with a shared style (use_plot_style)	replaceable
ipykernel 7.4.0	Jupyter kernel	lets VS Code run '# %%' cells in the Interactive Window	convenience
VS Code	editor	cell-by-cell running; .vscode/settings.json selects .venv	convenience
HTML (self-built)	results page	Report + build_results_page: one page with every result and figure	convenience
git (planned)	version control	.gitignore exists; not yet initialised	recommended

8.1 How they interact

- NumPy arrays are the common currency: the data scripts write .npy files, TensorFlow trains on NumPy arrays directly, predictions come back as NumPy arrays for metrics and routing.
- TensorFlow is imported only inside model functions, so the data and baseline scripts start fast and even work without it.
- matplotlib figures are saved to PNG and embedded (base64) into the HTML page by `show()`.

8.2 File formats

Format	Used for
.gz text	Abilene raw weeks (read with np.loadtxt)
.xml	GÉANT matrices and topology (parsed with regular expressions)
.npy / .npz	processed arrays; scaler min/max
.keras + _history.json	trained models and their loss curves
.csv	metrics log
.json / .html / .log	per-script report parts; results page; console logs

8.3 Installation and configuration

```
from README.md (Windows)

py -3.13 -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
.venv\Scripts\python common.py              # prints status and runs self_test()
.venv\Scripts\python scripts\00_setup.py    # must end with "Phase 0 complete"
.venv\Scripts\python scripts\01_data.py     # once: builds processed/
```

- Raw data must be downloaded separately: Abilene from Yin Zhang's page (cs.utexas.edu/~yzhang/research/AbileneTM/), GÉANT from the TOTEM project (totem.info.ucl.ac.be/dataset.html).
- Hardware: CPU only; no GPU needed. Disk: about 2 GB for processed arrays, 150 MB for models.
- Environment variable `TRAFFIC_PROJECT` can override the project folder.
- Set `RETRAIN = True` at the top of a model script to force retraining.

9. Data Flow and Worked Examples

All three examples use **real values from the project's data and saved models**, extracted for this guide with a read-only script. Example 1 is one forecast; example 2 is one congestion onset; example 3 shows training state over time.

Example 1: one 5-minute forecast for the busiest flow

Initial state: test sample #3691, target time 'X22 day 6 11:20' (file clock), flow WASH→WASH (flow 143), horizon 1.

Stage 1: the input window (last hour)

Step	1	2	3	4	5	6	7	8	9	10	11	12
Mbit/s	292	413	330	387	368	290	274	298	273	281	273	335
scaled z	0.9327	0.9503	0.9389	0.9470	0.9445	0.9324	0.9295	0.9339	0.9295	0.9309	0.9294	0.9397

These 12 values are one column of the 12 × 144 input; the other 143 flows are in the other columns. Scaling uses this flow's training min 0 and max 19.845 (log units).

Stage 2: the network

Stage	What happens to the sample
Conv1D ×2	64 filters scan every 3-step window across all 144 flows → 12 × 64 pattern responses
MaxPooling	→ 6 × 64 (strongest response of each pair of steps)
LSTM 100 → LSTM 50	read the 6 steps in order → one 50-number summary
Dense 'correction'	→ 144 corrections, one per flow; for WASH→WASH (mean of seeds) about −0.0037 in scaled units
Add last step	0.9397 (last value) + correction → forecast in scaled units
Clip	within [scaled(0), scaled(9.92 Gbit/s)] = [0, 1.110]: no change here

Stage 3: three seeds, undo scaling, compare

Forecaster	Scaled	Mbit/s	Error vs true 286.7 Mbit/s
Persistence (last value)	0.9397	334.9	48.2
CNN-LSTM seed 42	0.9344	301.6	14.8
CNN-LSTM seed 43	0.9396	334.3	47.5 (almost pure persistence)
CNN-LSTM seed 44	0.9340	298.8	12.1
CNN-LSTM, 3-seed mean (in Mbit/s)	–	**311.6**	**24.8**
True value	0.9319	286.7	–

Why this output: the last reading (334.9) was a jump up from about 273–281 in the four steps before it, and two of the three models nudged the forecast down. A plausible reading (an interpretation, not tested for this sample) is that the network has learned that a single-step spike on this flow is often partly reversed; section 11.3 of the log notes many one-step spikes on this flow. The truth (286.7) did fall back. Note how seeds differ: seed 43 barely moved. Over all 144 flows in this sample, MAE was 1.86 Mbit/s for persistence and 1.71 for the CNN-LSTM.

> **NOTE This example was chosen to illustrate a gain**
> It is the sample, among moves of 20–60 Mbit/s on this flow, where the model gained most. Typical corrections are far smaller: the median absolute correction over all test entries is 0.0012 in scaled units. On average the model gains only about 1% (chapter 10).

Example 2: a congestion onset on the busiest link, 30 minutes ahead

Initial state: horizon 6 (30 min), threshold 50%, link DNVR→KSCY (9.92 Gbit/s, carries 26 flows). Onset at 'X22 day 4 17:35'. Each forecast in the table was made 30 minutes before the time shown.

Time	True util.	Persistence forecast	CNN-LSTM forecast	True event?	Warnings (P / M)
17:20	5.2%	5.5%	5.3%	no	no / no
17:25	15.2%	5.5%	5.3%	no	no / no
17:30	42.6%	5.3%	5.2%	no	no / no
17:35	52.8%	5.4%	5.2%	yes (onset)	no / no
17:40	48.7%	5.2%	5.1%	no	no / no
17:45	27.8%	5.2%	5.0%	no	no / no
17:50	52.4%	5.2%	5.0%	yes (new onset)	no / no
17:55	34.3%	15.2%	14.8%	no	no / no
18:00	6.7%	42.6%	41.8%	no	no / no
18:05	9.7%	52.8%	51.7%	no	yes / yes

What happened: LOSA→CHIN jumped from about 96 Mbit/s (the last value the 30-minute forecast could see) to 4,849 Mbit/s at the onset. That flow crosses five backbone links (LOSA–SNVA–DNVR–KSCY–IPLS–CHIN); on this link the load went from 5% to 53% of capacity in 15 minutes. In the table, P = persistence and M = the CNN-LSTM (3-seed mean).

1. At 17:35 the true load crossed 50%: an onset. Both forecasts, made at 17:05 when the link was at about 5%, said about 5%. Neither warns: **both miss** (FN).
2. At 18:05 the burst is over (9.7%), but persistence now repeats the 52.8% it saw at 17:35 and warns: a **false alarm** (FP). The CNN-LSTM also forecasts 51.7% (persistence + a tiny correction) and also gives a false alarm.
3. The second crossing at 17:50 (after a dip to 27.8%) is a new onset, and both forecasts miss it too.
4. For this link, over its 3 onsets in the test period, persistence warned 0 and the CNN-LSTM warned 0 in advance.

> **KEY POINT Why this output was produced**
> A burst that begins and grows within 15 minutes cannot be seen in data that ends 30 minutes earlier. A model that forecasts only 3% of each change (damping 0.03 at 30 minutes) behaves like persistence shifted in time. This one example shows the project's main link-level finding.

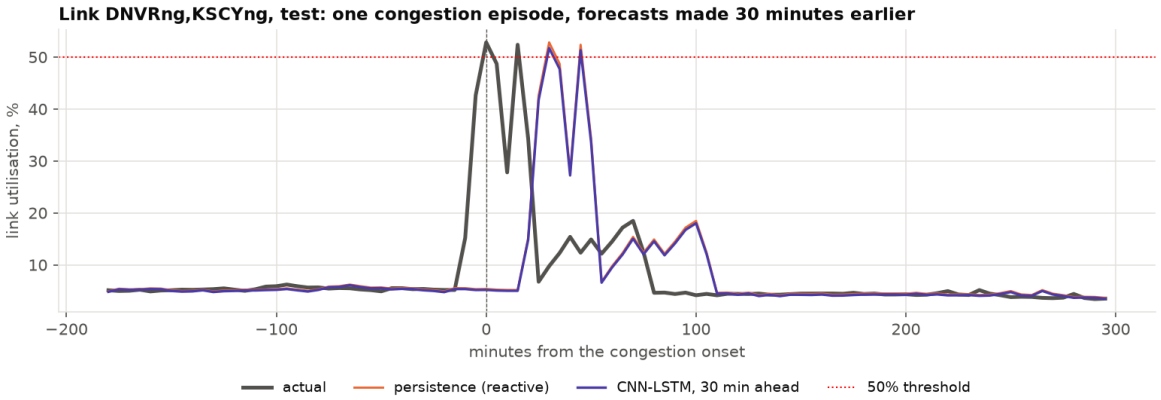


Figure: *The project's own plot of a congestion onset on the busiest link, 30-minute forecasts.* `(results/figures/04_episode.png)`

Example 3: how the training state changes over time

Model `p4_cnn_lstm_f64_k3_u100-50_d20_lr0.001_s42` (residual CNN-LSTM, original-units MAE loss, seed 42). Loss values are MAE divided by the mean training traffic (23.5 Mbit/s per flow).

Epoch	Train loss	Val loss	Learning rate	What is happening
1	0.1353	0.1163	0.001	starts at almost exactly persistence (val ≈ 2.742/23.5 = 0.1165)
5	0.1352	0.1163	0.001	hardly moving: the correction is still near 0
10	0.1349	0.1163	0.0005	no progress for 5 epochs → learning rate halved
20	0.1337	0.1149	0.0005	the correction starts to help
30	0.1328	0.1145	0.0005	slow, steady gains
40	0.1320	0.1142	0.00025	halved again
46	0.1316	**0.1140**	0.000125	**best epoch** (≈ 2.68 Mbit/s): checkpoint saved
61	0.1313	0.1141	0.00003	15 epochs without improvement → stop, restore epoch 46

Why training loss is higher than validation loss: the validation period is simply easier to forecast. Computed for this guide, persistence's own error in the same units is 0.1352 on the training split (3.18 Mbit/s), 0.1165 on validation (2.74) and 0.1365 on test (3.21). The model starts at persistence on both, so the gap is in the data, not a bug. Dropout, active only during training, adds a little more.

10. Testing, Evaluation and Results

10.1 How the project is tested

Test	What it checks	Where
self_test()	windows, split, scaler range and round trip, save/load, metrics, logging on synthetic data	common.py:1143, 00_setup
Model smoke test	the planned model builds with exactly 143,608 parameters; a forward pass gives (4, 144)	00_setup.py
Phase 1 exit check	contracted shapes and dtypes; X_train in [0, 1]; inverse_transform(y_test) equals the raw data	01_data.py §7
ARIMA alignment	ARIMA and persistence are scored on identical truth values (assert)	02_baselines.py
Residual start	an untrained residual model equals persistence exactly (checked in code, log §13.3)	PROJECT_LOG §13.3
Refactor check	prepare_abilene after refactoring onto _prepare_segments gives identical output	PROJECT_LOG §17.5
Seeds	3 seeds for every final model; spread reported	03c, 03d, 03e
Leakage rules	scaler on train only; chronological split; no window across gaps; no choices on test	by design

10.2 Important test scenarios

Type	Scenario	Handled by
Normal	a busy flow on an ordinary day	the whole pipeline
Boundary	an always-zero flow; a constant flow (range 0)	self_test zero flow; range_ = 1
Boundary	a segment shorter than 13 steps	skipped in windowing
Boundary	an onset right after a data gap	not counted (warn_onsets)
Invalid input	NaN or negative traffic into the scaler	ValueError
Invalid input	wrong file shape; missing file	ValueError / FileNotFoundError
Failure	input above the training range → extrapolation blow-up	forecast clipping in predict
Failure	script crashes	report part saved with the error

10.3 Measured results (all from the project's logs)

Baselines and single models (Abilene, test, 5 minutes)

Model	MAE Mbit/s	vs persistence	Flows better than persistence
Persistence	3.212	–	–
ARIMA (5 busiest flows only)	21.87 vs 21.28	−2.8%	1 of 5
LSTM-only, plain (MSE)	9.076	2.8× worse	2 of 144
CNN-only, plain (MSE)	10.821	3.4× worse	2 of 144
LSTM-only, residual (MSE)	3.226	−0.4%	48 of 144
CNN-only, residual (MSE)	3.222	−0.3%	37 of 144

Model	MAE Mbit/s	vs persistence	Flows better than persistence
CNN-LSTM, residual, MAE-in-Mbit/s loss (3 seeds)	**3.200 ± 0.005**	**+0.4%**	3 of 3 seeds beat persistence
LSTM-only, same settings (3 seeds)	**3.190 ± 0.010**	**+0.7%**	3 of 3 seeds

Horizons (test, mean of 3 seeds, % better than persistence)

Network, horizon	Persistence MAE	CNN-LSTM	LSTM-only	CNN-only
Abilene 5 min	3.212	+0.4%	+0.7%	+0.3%
Abilene 30 min	5.300	+0.8%	−12.3% (0/3 seeds)	+0.9%
Abilene 60 min	6.157	+1.1%	−12.4% (0/3)	+0.9%
GÉANT 15 min	1.469	+0.3%	+0.3%	+0.1%
GÉANT 60 min	2.377	**+2.7%**	+1.4%	+1.4%

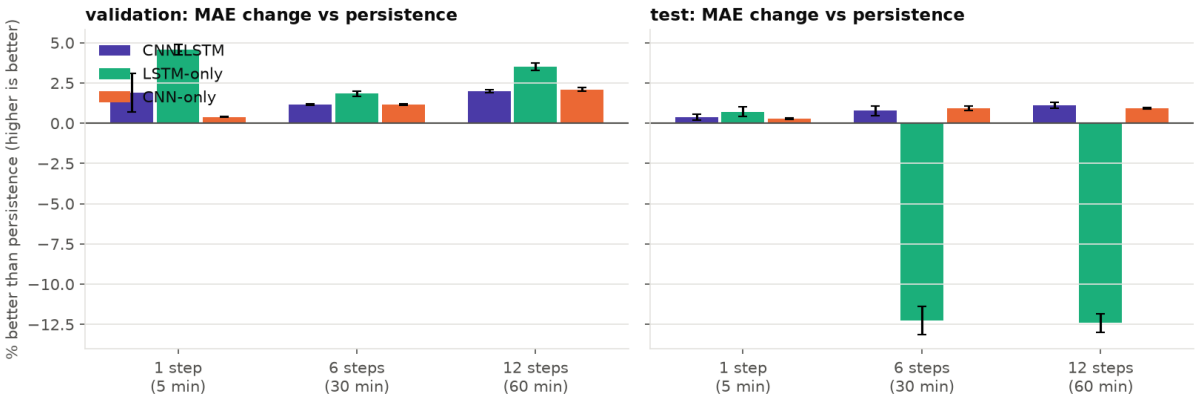


Figure: *Abilene: change versus persistence by horizon, validation (left) and test (right).* `(results/figures/03d_horizons.png)`

Ablations (Abilene, 1 seed, test % vs persistence)

Variant	5 min	30 min	60 min
Full CNN-LSTM (3-seed mean)	+0.4%	+0.8%	+1.1%
1 conv layer	+0.3%	+0.3%	0.0%
No pooling	+0.6%	+0.4%	−1.5%
1 LSTM layer	+0.7%	+0.4%	+1.1%

Link level (Abilene, test, 50% threshold)

Horizon	Util. MAE pers. / CNN-LSTM (pp)	F1 pers. / CNN-LSTM	Onsets warned in advance (both)
5 min	0.203 / 0.204	0.97 / 0.97	0 of 35
30 min	0.381 / 0.386	0.93 / 0.94	5 of 35
60 min	0.494 / 0.502	0.90 / 0.91	0 of 35

At 60% and 60 minutes the CNN-LSTM warned 9 of 26 onsets against persistence's 12. On GÉANT (approximate routing, relative threshold) the CNN-LSTM had higher precision (0.84 vs 0.76) but lower recall (0.65 vs 0.76) at 60 minutes, so F1 was no better.

Error analysis (test, 3-seed mean forecast, % better than persistence by size of the true move)

Move size	Abil. 5 min	Abil. 30 min	Abil. 60 min	GÉANT 15	GÉANT 60
smallest 50%	−3.9%	−8.7%	−13.2%	−7.7%	−16.1%
50–80%	+1.3%	+1.7%	+2.5%	−0.3%	+2.2%
80–95%	+2.3%	+3.0%	+4.4%	+2.2%	+6.5%
95–99%	+1.9%	+4.3%	+6.7%	+2.9%	+7.0%
top 1%	+1.2%	+3.8%	+5.6%	+2.9%	+7.0%

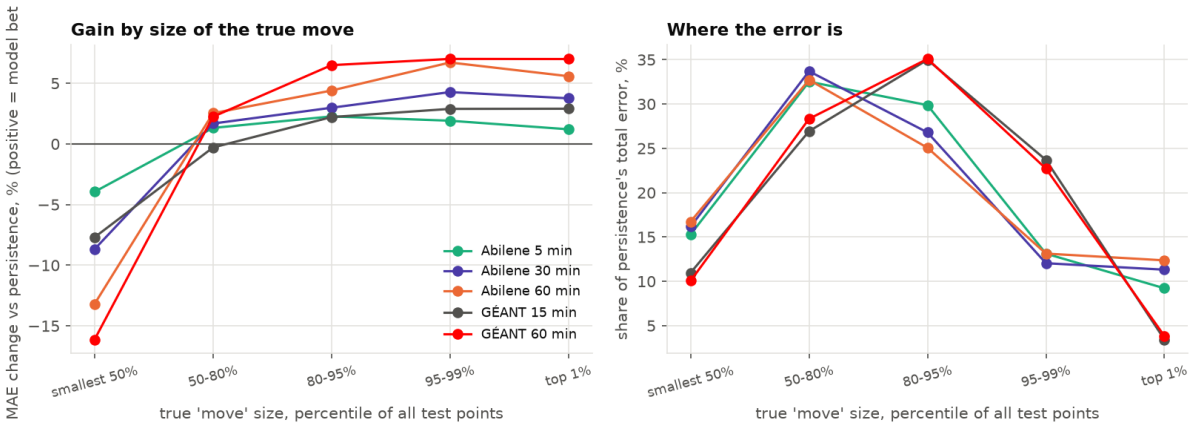


Figure: *Where the CNN-LSTM gains and loses against persistence, by the size of the real move.* (`results/figures/05_move_bins.png`)

10.4 What the results demonstrate

- The pipeline works end to end and is reproducible: data → models → link loads → warnings → report.
- Persistence is a very strong baseline for 5–15-minute backbone traffic.
- With a residual output and an original-units MAE loss, every final model beats persistence on flows, by about 0.3–1% at short horizons and 1–3% at 60 minutes. The gain grows with horizon on both networks.
- The gain comes from large moves; the model loses slightly on small moves and under-forecasts peaks more than persistence does.
- Better flow forecasts did not translate into better congestion warnings on either network.

10.5 What the results do NOT prove

- That CNN-LSTM is better than LSTM in general: on Abilene the ranking is decided by one flow, and on GÉANT the order differs.
- That the method would help a live network: everything is offline, on 2004–05 data, without acting on the warnings.
- Statistical significance in a formal sense: there are 3 seeds, one split and one test period per network; no confidence intervals or tests.
- Anything about GÉANT capacities: its link-level results use estimated routing and a relative threshold.
- That ablation differences of about 1% are real: they are single-seed, within the noise.

10.6 Known bugs, limitations and unverified claims

Item	Status
Forecast blow-up for out-of-range inputs	found and fixed (clipping in predict); Phase 3 / 03b numbers are without the fix
fit_or_load reuses stale models if code changes	by design; needs RETRAIN = True; documented
GÉANT absent pairs = 0	assumption, plausible but unverified
GÉANT time zone	unknown; the models do not use time of day, so results are unaffected
Abilene README dates	partly unreliable (duplicates found); handled by forcing segment breaks
Over-smoothing hypothesis (Phases 2–5)	tested in Phase 7 and found **wrong in direction**; replaced by the peak/mean-reversion finding
Unpinned requirements	versions not fixed in requirements.txt

11. Novelty and Design Decisions

11.1 What is genuinely distinctive

Combining a CNN and an LSTM for traffic forecasting is **not new**; such hybrids are common in the literature, and this project does not invent a new architecture. Its distinctive contributions are in **how it evaluates** and **what it found**:

1. **A strict, leak-free evaluation against persistence** at every horizon, on two networks, with seeds, so that a 1% gain can be told apart from noise.
2. **Evidence that the output form and loss matter more than the architecture:** plain models were 2.8–3.4× worse than persistence; residual output fixed most of it; training on MAE in original units was what finally beat persistence.
3. **An end-to-end check at link level**: flow-level gains of about 1% did *not* become better congestion warnings.
4. **A diagnosis of why**: damping of 1–5% (the model is persistence plus a small nudge); gains on large moves but losses on small ones; under-forecast peaks; one bursty flow (LOSA→CHIN) decides model rankings and 96% of congestion events.
5. **A correction of its own earlier hypothesis**: 'over-smoothing' was tested and found wrong in direction, and the record keeps both.

11.2 Existing methods it uses

Method	Origin	How the project differs in using it
Persistence, ARIMA	classical forecasting	used as the gate every model must pass
LSTM, 1-D CNN, CNN-LSTM	standard deep learning	compared in plain and residual forms, with ablations
Residual connections	e.g. ResNets; residual forecasting	applied at the output so the untrained model is persistence
Traffic matrices + routing matrix	network tomography / traffic engineering	used to go from flow forecasts to link warnings
Precision / recall / F1	classification	applied to congestion warnings, plus an onset-in-advance count

11.3 Key design decisions and their trade-offs

Decision	Benefit	Cost / trade-off
Forecast per OD flow, not per link	re-routable; matches the dataset	144 (or 529) outputs; link errors combine flow errors
Remove bad data, never fill	no invented values	15.6% of Abilene lost; 9 segments
Chronological split	realistic, leak-free	test period burstier than validation; rankings can flip
log1p + per-flow min–max	stable training across sizes	a scaled error ≠ a real error; needs an original-units loss
Residual output	starts at persistence; beats it	makes the model conservative (damping 1–5%)
MAE in original units as the loss	optimises the reported metric	big flows dominate; small flows matter less
Forecast clipping	no physical impossibilities	hides how badly a model extrapolates

Decision	Benefit	Cost / trade-off
One-at-a-time tuning, 0.5% rule	10 runs instead of 243	interactions between settings not explored
Three seeds	noise level is visible	3× the training time
50% threshold instead of 80%	events exist to score	a deviation from the plan, documented
Approximate GÉANT routing	allows a link-level check	not real congestion; approximate
Cells in .py files	diff-able, terminal-runnable	no saved outputs inside the files (they go to the report)

11.4 Implemented vs future work

Implemented: everything in chapters 4–10. **Future work:** seasonal baselines and time-of-day features, window tuning, ARIMA at longer horizons, a third dataset, acting on warnings (rerouting simulation), the final report and an updated presentation. See chapter 12.

12. Limitations and Future Improvements

12.1 Current limitations (existing functionality)

Area	Limitation
Accuracy	Gains over persistence are small: 0.3–1% at 5–15 min, 1–3% at 60 min. Forecast error is still 13–26% of the traffic on Abilene.
Peaks	Peaks are under-forecast more than persistence does (e.g. −21% vs −15% bias at 60 min, Abilene); the model reverts to typical levels.
Early warning	No advantage over reacting to current load on either network.
Inputs	Only the last hour of traffic. No time of day, day of week or calendar features, although the daily cycle is clear.
Data	2004–05 traffic; Abilene about 4.5 months of training data, GÉANT about 2.5; few weekly cycles to learn from.
Statistics	3 seeds, one split, one test period per network; no confidence intervals; ablations single-seed.
Dependence on one flow	Abilene rankings and 96% of congestion events come from LOSA→CHIN.
GÉANT link level	No capacities or IGP weights; hop-count routing; relative threshold.
Scalability	One output per OD pair grows as n^2: 144 → 529 flows nearly doubled the parameters (143,608 → 237,163). A network with 100 routers would have 10,000 outputs.
Resources	CPU-only is fine at this size; about 2 GB of processed arrays on disk.
Operations	Offline batch only: no streaming input, no retraining schedule, no monitoring of model drift.
Reliability	Forecasts are clipped, but out-of-range inputs (new traffic levels) still make the model unreliable; nothing detects drift.
Security	Not applicable to an offline research pipeline. In a live deployment the measurement feed would need integrity checks, because poisoned inputs could trigger false warnings.

12.2 Assumptions that may not hold in the real world

- Routing is fixed during the test period (Abilene A matrix from one date).
- Absent GÉANT pairs are near zero.
- Sampled NetFlow (1 in 100) scaled up is accurate enough for small flows.
- The past hour contains the information needed; bursts like LOSA→CHIN appear without warning in the data.

12.3 Suggested improvements (not implemented)

Improvement	How it could help	Effort
Seasonal naive baseline ('same time yesterday / last week')	a stronger, fairer reference; may itself beat persistence at 60 min	low
Time-of-day and day-of-week inputs	lets the model use the daily cycle it currently cannot see	low
Longer input window (e.g. 1 day)	more context for slow patterns	medium (memory)
More seeds (5–10) and confidence intervals	clear verdicts on 1% differences	medium (time)
Quantile or probabilistic forecasts	warn on 'chance of exceeding 50%' instead of a point forecast; better for bursts	medium

Improvement	How it could help	Effort
Train on link loads directly, or a link-aware loss	optimise what the warning uses	medium
Graph neural network over the topology	use which flows share links	high
Third dataset (CESNET-TimeSeries24, Milan mobile grid)	test generalisation on modern / spatial data	medium
Simulate acting on warnings (rerouting)	measure the operational benefit, not only accuracy	high
Pin library versions; git; CI running self_test	reproducibility	low

13. Important Questions and Answers

38 questions likely in a viva, review or interview, grouped by type. Answers are short enough to say aloud, and each one is backed by earlier chapters.

13.1 Basic

Q1. What does your project do, in one sentence?

It forecasts the traffic between every pair of backbone routers 5–60 minutes ahead with a CNN-LSTM, turns those forecasts into predicted link loads through the routing matrix, and checks whether that warns of congestion earlier than simply watching the current load.

Q2. What is an origin–destination flow?

All traffic entering the network at router i and leaving at router j. Abilene has 12 routers, so 144 flows; GÉANT has 23, so 529.

Q3. Why not forecast link loads directly?

Flows are what the dataset measures, and a flow forecast can be mapped onto any routing (load = A @ flows). If routing changes, the same flow forecasts still apply. The cost is that link errors combine many flow errors.

Q4. What is persistence and why is it so important?

Forecasting that the next value equals the current one. At 5-minute steps traffic changes little, so it is very accurate. Any model must beat it to be useful, so every result is reported against it.

Q5. What dataset did you use?

Abilene (US research backbone, 12 routers, 144 flows, 5-minute readings, 24 weeks, March–September 2004, measured 'realOD' column) and GÉANT (European, 23 routers, 529 flows, 15-minute readings, January–April 2005, TOTEM dataset). Both are real, historical measurements, used offline.

13.2 Concepts

Q6. Why combine a CNN and an LSTM?

The idea was that causal 1-D convolutions pick up short patterns (like a rise over three readings) across flows, and the LSTM models how those patterns evolve. In the results, the CNN mainly added robustness to out-of-range bursts on Abilene, and the hybrid was best on GÉANT at 60 minutes.

Q7. What does 'causal' padding mean, and why use it?

Each convolution output at time t only uses inputs at t and earlier. Otherwise the model could use later steps of the window, which would not be available for the step being represented.

Q8. What is a residual model here?

The network outputs a correction that is added to the last observed value. Its correction layer starts at zero, so an untrained model equals persistence. Training only moves it away from persistence where that lowers the loss.

Q9. Why log-transform the data?

Values range from 0 to billions of raw units, and a few flows are thousands of times larger than others. The log compresses the range so training is stable; log1p keeps zero valid.

Q10. Why fit the scaler on training data only?

Using min/max from the whole dataset would leak information about the test period into training and make results optimistic.

13.3 Architecture

Q11. Walk me through the pipeline.

Load 24 weeks → keep realOD → clean with rules R1–R4 (84.4% kept) → cut into 9 gap-free segments → 1-hour windows → chronological 60/20/20 split → log + min–max scaling fitted on train → train residual CNN-LSTM on MAE in original units with early stopping → predict, clip, unscale → metrics against persistence → link utilisation via A → warnings scored with precision, recall, F1 and onsets → results page.

Q12. How do the scripts share code?

Everything reusable is in common.py, and each script finds and imports it. Scripts communicate only through files with a fixed contract: .npy arrays, scaler.npz, .keras models and metrics.csv.

Q13. What are the layers and how many parameters?

Conv1D(64, 3, causal) ×2 → MaxPooling(2) → LSTM(100) → LSTM(50) → Dropout(0.2) → Dense(144) correction, plus the skip path. 143,608 parameters on Abilene, 237,163 on GÉANT.

13.4 Implementation and code decisions

Q14. Why did the first models do so badly?

Predicting each flow's level from scratch, they pulled forecasts toward typical values and missed sustained new levels and bursts, giving 2.8–3.4× persistence's error. Persistence simply repeats the last value.

Q15. What was the single most important change?

Two together: the residual output (from 3× worse to level with persistence) and training on MAE in original units (to actually beat it). On the log scale every flow counts equally, so the model optimised flows that hardly matter in Mbit/s.

Q16. How did you stop overfitting?

Dropout 0.2; early stopping on validation loss with best-weight restore; learning-rate halving on plateaus; no choices made on the test split.

Q17. How did you tune hyper-parameters?

One setting at a time from the planned values (filters, kernel, LSTM units, dropout, learning rate), validation only, keeping a change only if it improved validation MAE by more than 0.5%. Nothing passed, so the planned settings stayed.

Q18. Why three seeds?

The gains are about 1% and the seed-to-seed spread is 0.5–1%. Without seeds you cannot tell a real gain from luck.

Q19. What went wrong with the forecast blow-up and how was it fixed?

A validation input above the training range made one model output 1.99 in scaled units; the inverse exponential turned that into about 25 million Mbit/s. predict now clips forecasts between zero and the fastest link's capacity.

Q20. How do you handle missing data?

Rules: remove duplicated days, day-long outages of normally active flows, hour-long router outages and impossible values; keep normal zeros and real bursts; never fill; cut into segments so no window crosses a gap.

13.5 Results

Q21. How much better than persistence is your model?

On flows: about +0.4% at 5 min, +0.8% at 30 min, +1.1% at 60 min on Abilene; +0.3% at 15 min and +2.7% at 60 min on GÉANT. Forecast error is about 13–26% of the traffic on Abilene.

Q22. Did the model give earlier congestion warnings?

No. On Abilene F1 was 0.97 / 0.94 / 0.91 against persistence's 0.97 / 0.93 / 0.90, with the same or fewer onsets warned in advance. On GÉANT it gave fewer false alarms but missed more events.

Q23. Why is F1 so high for persistence?

Congestion episodes last many steps. Persistence is correct on every step of an episode except where it starts and ends.

Q24. Why did LSTM-only fail at 30 and 60 minutes on Abilene?

Entirely because of LOSA→CHIN, whose test bursts (inputs up to 1.28 scaled) exceed anything in training. The LSTM extrapolated; without that flow it was as good as or better than the CNN-LSTM.

Q25. Why use 50% and not 80% as the congestion threshold?

No Abilene link ever reaches 80% (the busiest reaches 75%), so 80% has no events to score. 50% was fixed before looking at warning results, with 40% and 60% as checks. It is documented as a deviation.

13.6 Testing and metrics

Q26. How did you test the code itself?

self_test runs the data pipeline on synthetic data with assertions; 00_setup checks the model size and a forward pass; 01_data checks shapes and the scaling round trip against raw data; 02_baselines asserts ARIMA and persistence are scored on identical truth.

Q27. What metrics did you use and why?

MAE (main; in Mbit/s), RMSE (sensitive to bursts), MAPE with coverage (only above 10% of the mean, otherwise near-zero flows explode it), % change versus persistence, and precision/recall/F1/onsets for warnings.

13.7 Novelty

Q28. What is novel in your project?

Not the architecture. The contribution is a strict, leak-free, end-to-end evaluation on two networks, the finding that output form and loss matter more than architecture, and the diagnosis of why flow gains do not become better warnings.

13.8 Limitations and next steps

Q29. What are the main limitations?

Small gains; no time-of-day input; old and short datasets; 3 seeds only; one flow dominates Abilene; GÉANT link level is approximate; offline only.

Q30. What would you do next?

Add a seasonal baseline and time-of-day features, try a longer window, more seeds, probabilistic forecasts for exceedance warnings, a third dataset, and simulate acting on the warnings.

13.9 Challenging questions

Q31. Your gain is 0.4%. Isn't that just noise?

At 5 min the per-seed spread is ±0.005 Mbit/s on 3.2, and all 3 seeds beat persistence, so it is consistent but tiny. At short horizons I would not claim practical value. At 60 minutes on GÉANT (+2.7%, all seeds) the gain is clearer.

Q32. If the model is basically persistence, what has it learned?

Small, systematic corrections: it partly anticipates large moves (gaining 4–7% on them at 60 min) and pays for it on small moves. Damping of 0.01–0.05 means it forecasts only 1–5% of each move, because most 5–15-minute variation is unpredictable from the last hour.

Q33. Why can persistence never warn of an onset 5 minutes ahead?

An onset at t means the load at t−1 was below the threshold, and persistence's forecast for t is exactly the load at t−1. A model that is persistence plus a tiny correction inherits this.

Q34. Doesn't shuffling the training batches leak information?

No. Shuffling happens only inside the training split; every window there is earlier than every validation and test window. Leakage comes from mixing splits, which the chronological split prevents.

Q35. Why not just use RMSE or MSE as the loss?

MSE on scaled data was tried: validation MAE 2.777 versus persistence 2.742, so it was worse. The model should optimise the metric it is judged on; MAE in original units did, and gave 2.693.

Q36. Why does the model under-forecast peaks more than persistence?

Measured: peak bias is −21% for the CNN-LSTM vs −15% for persistence (Abilene, 60 min). The project's interpretation is mean reversion: at a peak the correction pulls toward typical levels, which pays off after a one-step spike but not while a high level persists. It matches the link-level behaviour: higher precision, lower recall.

Q37. Could your results be due to the specific test period?

Partly. The Abilene test period is burstier than validation, and rankings flipped between them. That is why a second network was added; the stable finding across both is 'small gain that grows with horizon'. Model rankings are not stable.

Q38. Why is training loss higher than validation loss?

The validation period is easier. Persistence's own error is 0.135 on train vs 0.117 on validation in the loss units, and the model starts from persistence. Dropout also only applies during training.

14. Quick Revision

14.1 One-page summary

Question	Answer
Goal	Forecast backbone OD traffic 5–60 min ahead; map to link loads; warn of congestion before it happens.
Data	Abilene 144 flows × 5 min × 24 weeks (2004); GÉANT 529 flows × 15 min (2005). Real measurements, offline.
Preparation	realOD; rules R1–R4 (84.4% kept); 9 segments; 12-step windows; 60/20/20 by time; log1p + min–max on train.
Model	Residual CNN-LSTM: Conv1D×2 → Pool → LSTM 100 → LSTM 50 → Dropout → Dense correction + last value. 143,608 params.
Training	Adam 1e-3, batch 128, MAE in original units, early stopping, LR halving, 3 seeds.
Flow result	Beats persistence by +0.4% (5 min) to +1.1% (60 min) on Abilene, +2.7% at 60 min on GÉANT.
Link result	No early-warning advantage (Abilene F1 0.97 / 0.94 / 0.91 vs 0.97 / 0.93 / 0.90).
Why	Short-term moves are largely unpredictable; the model forecasts 1–5% of each move; peaks revert; one flow dominates.
Contribution	Rigorous evaluation; output form and loss matter more than architecture; flow gains do not become warnings.

14.2 The complete flow in numbered steps

1. Read 24 weekly files; keep realOD.
2. Remove duplicates, outages and impossible values.
3. Cut the kept data into gap-free segments.
4. Make 12-step input windows and targets h steps ahead.
5. Split by time 60/20/20.
6. Fit log + min–max on training inputs; scale everything.
7. Build the residual CNN-LSTM (correction starts at 0).
8. Train on MAE in original units; stop on validation.
9. Predict; clip to 0..link capacity; undo the scaling.
10. Score MAE/RMSE/MAPE against persistence; log to metrics.csv.
11. Map flows to link utilisation with A; warn at ≥ 50%.
12. Score warnings (P/R/F1, onsets); analyse errors; build report.html.

14.3 Concepts to remember

- Persistence is the bar.
- Chronological split; scaler on train only; no window across a gap.
- Residual = last value + correction.
- Loss in the units you report.
- Causal convolution.
- Link load = A @ flows; utilisation = load / capacity.
- Onset = first congested step after a non-congested one.

- Damping = share of a real move that is forecast.

14.4 Key formulas and metrics

Name	Formula
Scaling	$z = (\log(1+x) - \min) / (\max - \min)$; inverse $x = \exp(z \cdot \text{range} + \min) - 1$
Mbit/s	raw × 100 × 8 / 300 / 10^6
Samples per segment	$n = T - W - h + 1$
MAE / RMSE	mean $\lvert e \rvert$ / sqrt(mean e^2)
MAPE	mean $\lvert e \rvert$ / true over true ≥ 0.1 × mean(true); report coverage
Gain vs persistence	$(1 - \text{MAE}_{\text{model}} / \text{MAE}_{\text{pers}}) \times 100\%$
Utilisation	(A @ flows) / capacity
Precision / recall / F1	TP/(TP+FP), TP/(TP+FN), 2PR/(P+R)
Damping	Σ (forecast − last)(true − last) / Σ (true − last)^2
Conv1D params	k × in × filters + filters
LSTM params	4 × (u × (in + u) + u)

14.5 Component → responsibility

Component	Responsibility
common.py	all shared logic
01_data.py	data preparation
02_baselines.py	persistence, ARIMA
03_models.py	LSTM, CNN (plain/residual)
03b_hybrid.py	CNN-LSTM, loss comparison
03c_tuning.py	tuning, seeds
03d_horizons.py	horizons, ablations
03e_geant.py	second network
04_analysis.py	link loads, warnings
05_errors.py	error analysis
results/report.html	everything on one page

14.6 Most important functions

Function	One line
FlowScaler	log1p + per-flow min–max, and its inverse
make_windows	series → (X, y) samples
abilene_clean / geant_clean	missing-data rules → segments
_prepare_segments	window, split, scale, save
build_cnn_lstm + _output	the network, residual or plain
make_loss('mae_raw')	MAE in original units
fit_or_load / train_model	train with early stopping, or reuse

Function	One line
predict	forecasts clipped to the physical range
evaluate	MAE, RMSE, MAPE + coverage
load_abilene_routing	routing matrix A and capacities
warn_prf / warn_onsets	warning quality and onsets
Report / build_results_page	the results page

14.7 Novelty in two sentences

The project does not invent a new model. It shows, with a strict leak-free evaluation on two real backbone networks, that a CNN-LSTM beats persistence only when built as a residual on persistence and trained on the real-unit error, and that even then its ~1–3% flow-level gain does not produce earlier congestion warnings, and it explains why.

14.8 The two-minute explanation

2 MIN Say this

My project forecasts traffic on a backbone network so that congestion can be flagged before it happens. On the Abilene research network, 12 routers exchange traffic, so there are 144 router-to-router flows, measured every 5 minutes for 24 weeks.

I cleaned the data with fixed rules, never inventing values, cut it into 1-hour windows, split it by time and scaled it using the training data only. The model is a CNN-LSTM: convolutions find short patterns in the last hour, LSTMs model how they evolve.

The key comparison is persistence, 'the next 5 minutes look like the last 5'. It is very hard to beat. My first models were three times worse. Two changes fixed that: predicting a correction to the last value instead of the value itself, and training on the error in real units. With those, the CNN-LSTM beats persistence on both Abilene and the European GÉANT network, by about 0.4% at 5 minutes and up to 2.7% at an hour.

I then mapped the forecasts onto the 30 network links and scored congestion warnings. Here the model gives no advantage over reacting to the current load. My error analysis shows why: the model forecasts only a few percent of each real change, because short-term traffic bursts are mostly not predictable from the last hour, and one bursty flow causes almost all congestion.

So the contribution is a careful, reproducible evaluation showing what helps, what doesn't, and why. Next steps are time-of-day features, seasonal baselines and probabilistic forecasts.

Glossary

Term	Meaning
Ablation	Removing one part of a model to see how much it contributes.
Abilene	US research backbone network (Internet2); 12 routers in this dataset.
Adam	An optimiser: gradient descent with adaptive per-weight step sizes.
ARIMA	AutoRegressive Integrated Moving Average: a classical statistical forecaster.
Backbone link	A high-capacity link between core routers (rows 1–30 of Abilene's A).
Baseline	A simple reference method that any proposed model must beat.
Batch / epoch	A batch is a group of samples per weight update (128); an epoch is one pass over all training samples.
Causal padding	Convolution padding that lets an output use only current and earlier inputs.
Chronological split	Splitting data by time order, never randomly.
CNN / Conv1D	Convolutional network; Conv1D slides learned filters along time.
Coverage (MAPE)	Share of values included in MAPE after the floor (true ≥ 10% of the mean).
Damping	Slope of forecast change on true change; the share of each move that is forecast.
Data leakage	Information from the validation/test period reaching training, giving optimistic results.
Dense layer	Fully connected layer: weighted sum of all inputs plus a bias.
Dropout	Randomly zeroing a share of units during training to reduce overfitting.
Early stopping	Stopping training when validation loss stops improving.
F1 score	Harmonic mean of precision and recall.
Floyd–Warshall	All-pairs shortest-path algorithm, $O(n^3)$.
GÉANT	European research backbone; 23 routers (anonymised) in the TOTEM dataset.
Horizon	How many steps ahead a forecast is made.
Hyper-parameter	A setting chosen before training (filters, units, learning rate…).
Learning rate	The size of each weight update.
log1p / expm1	log(1 + x) and its inverse exp(x) − 1; safe for zero.
LSTM	Long Short-Term Memory: a recurrent layer with gated memory.
MAE / RMSE / MAPE	Mean absolute error / root mean squared error / mean absolute percentage error.
Max pooling	Keeping the maximum of each group of neighbouring values.
Mbit/s	Megabits per second (10^6 bits/s).
Min–max scaling	Linear rescaling so the training minimum is 0 and the maximum is 1.
NetFlow sampling	Routers record statistics on 1 in N packets (N = 100 on Abilene).
OD flow	Origin–destination flow: all traffic from ingress router i to egress router j.
Onset	The first congested step after a non-congested step on a link.
Overfitting	Learning the training data's noise so performance on new data drops.
Persistence	Forecast = most recent observed value.
Precision / recall	Share of warnings that were right / share of events that were warned.
realOD	Abilene's measured OD traffic column (the others are estimates).
ReLU	Activation max(0, x).
Residual model	Model that predicts a correction added to the last value.

Term	Meaning
Routing matrix (A)	Links × flows matrix of the share of each flow on each link.
Seed	Starting value of the random generator; same seed → same run.
Segment	A stretch of truly consecutive kept time steps; windows never cross segments.
Tomogravity	A method that estimates OD flows from router totals; Abilene columns 2–5 are such estimates.
Traffic matrix	Table of all OD flows at one time.
Utilisation	Link load divided by link capacity.
Validation split	Data used to choose settings and stop training; not for final scores.
WAPE	Weighted absolute percentage error: Σ|error| / Σ true.
Window	The input history given to the model: 12 steps here.

References to project sources

Topic	Where to read more
Data, cleaning, splits	PROJECT_LOG §10; common.py:303–489; scripts/01_data.py
Baselines	PROJECT_LOG §11; scripts/02_baselines.py
Single and residual models	PROJECT_LOG §13; common.py:703–761; scripts/03_models.py
Loss, tuning, seeds	PROJECT_LOG §14; common.py:764–851; scripts/03b_hybrid.py, 03c_tuning.py
Horizons, ablations	PROJECT_LOG §15; scripts/03d_horizons.py; results/03d_run.log
Link loads, warnings	PROJECT_LOG §16; common.py:632–697; scripts/04_analysis.py; results/04_run.log
GÉANT	PROJECT_LOG §17; common.py:494–629; scripts/03e_geant.py; results/03e_run.log
Error analysis	PROJECT_LOG §18; scripts/05_errors.py; results/05_run.log
All results	results/metrics.csv; results/report.html
Abilene dataset	raw/readme.txt; cs.utexas.edu/~yzhang/research/AbileneTM/
GÉANT dataset	totem.info.ucl.ac.be/dataset.html